Politécnico
de Viseu

Escola Superior
de Tecnologia
e Gestão de Viseu

AI-Enhanced Semantic Analysis of Vehicle Logs for Efficient Issue Tracking

Ricardo Almeida Pombo

Trabalho de Projeto

Licenciatura

Engenharia Informática

Junho 2025

Politécnico de Viseu

Escola Superior de Tecnologia e Gestão de Viseu

departamento de informática

AI-Enhanced Semantic Analysis of Vehicle Logs for Efficient Issue Tracking

Ricardo Almeida Pombo

Completed at Critical Software

Project Report

Bachelor's in

Computer Sciences Engineering

Work carried out under the supervision of
Carlos Augusto da Silva Cunha (ESTGV)
Joaquim Tojal (Critical Software)

June 2025

Acknowledgements

The realization of this internship would not have been possible without the guidance and support of several people and entities, to whom I leave my sincere thanks.

First of all, I would like to express my gratitude to Critical Software for the opportunity to carry out this internship, providing a stimulating, challenging and enriching professional environment, where I was able to deepen and apply my knowledge in artificial intelligence, machine learning and software development.

I would also like to thank my teammates and employees at Critical Software, who contributed directly and indirectly to the realization of this project, whether through technical suggestions, knowledge sharing or collaboration in the development process.

I also express my gratitude to my internship supervisor at the company, for the constant monitoring, availability and valuable suggestions that helped me to face the challenges with confidence, as well as to my academic supervisor, for his availability and monitoring, constructive feedback and support during the development of this work.

Finally, I leave a special thanks to my family and friends, for their unconditional support, understanding and encouragement throughout the academic path, especially during the most demanding moments of this project.

Abstract

This project was developed within the scope of the Project Curricular Unit of the Computer Engineering course at the Polytechnic Institute of Viseu, and took place from February to June at the company Critical Software.

The work consisted of the development of an intelligent tool for analyzing the system's feedback from tests in the automotive industry. The system aims to assist technical teams in the detection and interpretation of critical events, through automatic analysis and semantic search on the data.

A Machine Learning pipeline was implemented that integrates supervised classification models, such as Random Forest and neural networks (LSTM), and a clustering algorithm (DBSCAN), with a view to detecting and grouping error patterns, such as failures. At the same time, a semantic search system based on the Retrieval-Augmented Generation technique was also developed, using a local Large Language Model and vectors generated with semantic embeddings and indexed with Hugging Face. The system architecture follows the Model-View-Controller pattern, being divided into multiple containers.

The final system allows log file loading, automatic analysis with trained models, pattern visualization and natural language search, offering effective support for the analysis of large volumes of data.

Keywords: Machine Learning; Supervised; Random Forest; Neural Networks; Unsupervised; Retrieval-Augmented Generation.

Resumo

Este projeto foi desenvolvido no âmbito da Unidade Curricular de Projeto do curso de Engenharia Informática do Instituto Politécnico de Viseu, e decorreu desde fevereiro a junho na empresa Critical Software.

O trabalho consistiu no desenvolvimento de uma ferramenta inteligente para análise do feedback do sistema proveniente de testes na indústria automóvel. O sistema tem como objetivo auxiliar as equipas técnicas na deteção e interpretação de eventos críticos, através de análise automática e pesquisa semântica sobre os dados.

Para isso, foi implementado uma *pipeline* de *Machine Learning* que integra modelos de classificação supervisionada, como o *Random Forest* e redes neuronais (LSTM), e um algoritmo de *clustering* (DBSCAN), com vista à deteção e agrupamento de padrões de erro, como falhas.

Paralelamente, foi desenvolvido um sistema de pesquisa semântica baseado na técnica *Retrieval-Augmented Generation*, utilizando uma *Large Language Model* local e vetores gerados com *embeddings* semânticos e indexados com o *Hugging Face*. A arquitetura do sistema segue o padrão *Model-View-Controller*, estando dividida em múltiplos *containers*.

O sistema final permite o carregamento de ficheiros de *logs*, análise automática com modelos treinados, visualização de padrões e pesquisa em linguagem natural, oferecendo um apoio eficaz à análise de grandes volumes de dados.

PALAVRAS-CHAVE: Machine Learning; Supervisionada; Random Forest; Redes Neuronais; Não Supervisionada; Retrieval-Augmented Generation.

Índice

Índice de Figuras

Lista de siglas / abreviaturas

ML – Machine Learning

DBSCAN – Density-Based Spatial Clustering of Applications with Noise

LLM – Large Language Model

RAG – Retrieval-Augment Generation

API – Application Programming Interface

RAM – Random Access Memory

FAISS – Facebook AI Similarity Search

CNN – Convolutional Neural Networks

RNN – Recurrent Neural Networks

LSTM – Long Short-Term Memory

NPL – Natural Language Processing

1 Introduction

This internship report aims to describe and reflect on the experience acquired during the curricular internship period, fitting into the study plan of the Degree in Computer Engineering. The internship carried out at Critical Software took place in the 2025 academic year, starting on February 17th and ending on June 30th.

During the internship, an automatic log analysis tool (system feedback) was developed using Machine Learning and language models (LLM) techniques. This tool aims to detect, classify and group relevant events in log files from the tests carried out on car software, allowing the identification of error patterns and supporting teams in the analysis of the results.

Analyzing large amounts of data manually is a complicated task, which requires a lot of time and knowledge of what is being analyzed, and this can lead to errors and is not very scalable. For this reason, the need arose to develop an automated solution for this process, making it more accurate, efficient and usable in other testing contexts. The choice for ML techniques is related to their ability to identify complex patterns in the data and learn from examples. The LLM implemented also contributes to the search for information.

The main objective of this work was to create a tool that would provide technical support to the team in the detection of important events in the system data, allowing the identification of failures in the tests. The project also served to apply knowledge acquired during academic training, as well as to develop technical and interpersonal skills, and gain some experience in the business environment.

This tool was designed by building an ML pipeline divided into several steps, pre-processing the data, training supervised and unsupervised models, reinforcement

learning (namely Random Forest, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Neural Networks), and applying the models to new test data. Each step gave rise to an output file that served as input for the next, enabling reuse and modularity. To simplify the team's tasks and improve the tool, a local Application Programming Interface (API) was created with a web interface (Streamlit) to be able to upload and analyze the processed files. It also involved the incorporation of an LLM, using a RAG approach, with the potential to answer users' questions about the data.

1.1 Host Entity

The host entity was Critical Software. It is a Portuguese multinational technology company specializing in the development of software solutions and engineering services for safety, mission, and business-critical applications [1]. Founded in 1998 by three engineers from the University of Coimbra, the company originated from the Instituto Pedro Nunes (IPN), a business incubator and technology transfer center. Headquartered in Coimbra, Portugal, Critical Software has expanded globally with offices in Porto, Lisbon, Viseu, Southampton (UK), Munich (Germany), and Boston (USA) and hubs in Algarve, Tomar and Vila Real.

Core Competencies and Services

Critical Software provides a wide range of services across diverse industries such as aerospace, defense, automotive, railway, telecoms, finance, energy, and utilities. Its expertise includes:

- System Design and Development: Embedded and real-time systems, command and control systems.
- Software Verification & Validation: Ensuring compliance with stringent safety standards.
- Digital Transformation: AI-driven solutions and smart meter testing.
- Security and Infrastructure: Cybersecurity measures for critical systems.

The company is renowned for its contributions to high-integrity software development. It was the first delivery unit in Portugal to achieve CMMI Maturity Level 5 certification for both waterfall and agile methodologies.

Industry Contributions

Critical Software has made significant advancements in various sectors:

- Aerospace: Developing embedded software for critical systems and ensuring compliance with strict standards like RAMS analysis.
- Automotive: Creating software for electronic control units and autonomous driving systems while addressing cybersecurity challenges.
- Finance: Modernizing banking systems and enabling digital transformation through innovative FinTech solutions.
- Defense: Building mission-critical applications such as command systems and training simulators.

Achievements and Culture

The company's commitment to quality is evident in its collaborations with prestigious organizations like NASA and the European Space Agency (ESA). Its software solutions are trusted globally, from orbiting satellites to powering reliable smart meter networks on Earth. Critical Software emphasizes a culture of innovation, reliability, and excellence while tackling complex technological challenges.

Global Presence

With over 1,300 employees across seven offices worldwide, Critical Software continues to deliver cutting-edge solutions that transform industries. Its multidisciplinary approach ensures adaptability to client needs while maintaining rigorous standards for safety and performance

1.2 Objectives and Expected Results

The objective of this internship is to develop an automatic log analysis tool using Machine Learning techniques, aimed at interpreting, analyzing, and processing logs from automotive infotainment systems. The goal is to categorize, classify, and

investigate events resulting from test-induced actions within the automotive industry, thereby reducing the manual effort required to analyze large volumes of data. Building upon this, the project was structured around several concrete objectives, from data preparation to the creation of an accessible and intelligent interface, as detailed below.

The project has the following main objectives:

- **Analyze the available data**, with a particular focus on identifying structures, patterns, and relevant information within log files generated during system operations. This includes understanding the context in which the logs are produced and the type of events they represent.
- **Pre-process and prepare the data** to ensure its quality and usability for Machine Learning applications. This involves cleaning irrelevant or redundant information, standardizing formats, extracting features, and transforming textual data where necessary, particularly from the 'Payload' field.
- **Develop and train Machine Learning models** capable of detecting both errors and patterns within the logs. This includes classification models (e.g., Random Forest) for error identification and unsupervised learning algorithms (e.g., DBSCAN) for clustering similar behaviors or anomalies.
- **Design and implement a modular and reusable processing pipeline**, allowing the trained models to be applied to new and unseen data in an automated way. The modularity ensures that the solution can adapt to different types of logs and system outputs without significant reconfiguration.
- **Build an API with a web interface**, enabling users to upload log files, trigger the analysis pipeline and interact with the results directly throught the browser. This component is essential to ensure accessibility, usability, and integration in real-world workflows.

- **Create an interactive analysis tool** connected to the API, integrating a Language Model (LLM) for natural interaction with the processed data. This tool facilitates a more intuitive and accessible way for users to query, interpret, and understand the insights extracted from the logs.

It is expected that the developed solution will significantly improve the efficiency of analyzing system feedback, enable early detection of failures or irregular behaviors, and establish a robust and extensible technological foundation. This foundation not only supports the continued enhancement of the current tool but also provides a reusable architecture for future projects with similar requirements in the domain of intelligent log analysis and system monitoring.

1.3 Planning

During the internship period, a work plan was defined, divided into tasks with the respective assigned duration. These tasks were organized in sequential stages, ensuring a structured and consistent development of the project. This organization

allowed not only a good management of progress but, also helped with the documentation and validation of the results obtained.

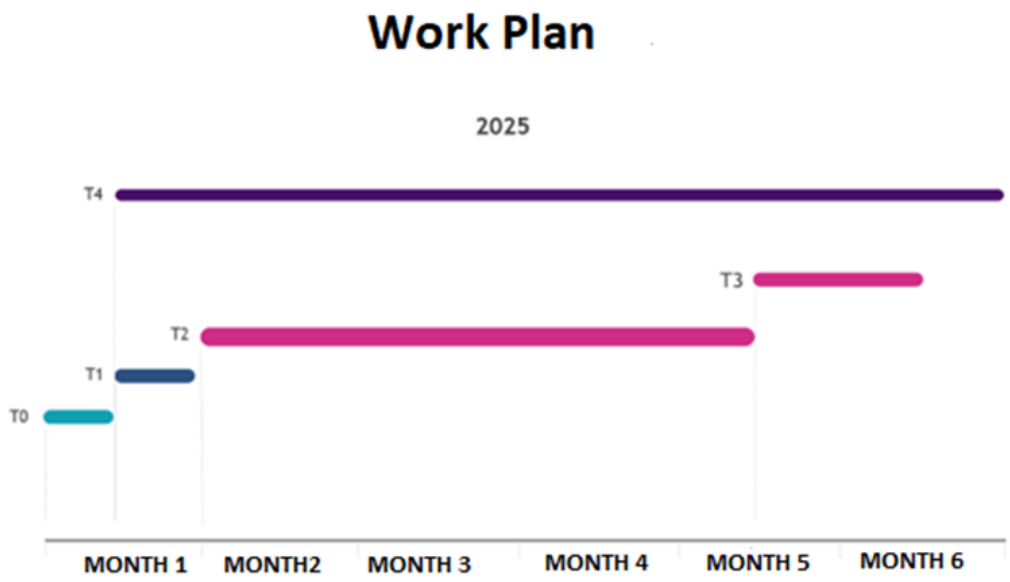

Figure 1 - Working Plan

T0: Familiarization with the automotive sector:

This stage corresponds to the initial planning phase, dedicated to familiarization with the automotive sector. For this, an in-depth analysis of the technical documentation of the AUTOSAR and COVESA standards was made, allowing to understand the requirements, terminologies and common practices of the area.

T1: Identification of the Machine Learning technology for the project:

Here, an analysis of the different Machine Learning technologies will have to be carried out to select which ones are most suitable for detecting and classifying patterns in logs in the automotive industry.

T2: Machine Learning Model Development:

During this period, the development of the ML models chosen in the previous phase is planned. The data will be used to train the models, adjusting their parameters based on appropriate performance metrics.

T3: Testing and Optimization:

At this time, tests will be carried out with real data extracted from the Electronic Control Units (ECUs) of the car entertainment system, to validate the performance of the developed models. The results obtained will serve as a basis for adjusting the parameters of the models, with the aim of improving their accuracy in finding patterns of unwanted behavior. The performance of the algorithms will also be optimized, ensuring their efficient processing capacity even in large amounts of data.

T4: Documentation and Presentation:

This is the last stage of planning, but it is developed throughout the internship. It corresponds to the gradual preparation of the technical documentation and the user manual. To ensure a rigorous and up-to-date record of the decisions, methodologies and implementation made. At the end of the project, the documentation will be consolidated, and presentations will be prepared, both for academic and internal evaluation at Critical Software.

1.4 Structure of the Report

The report is structured to clearly and logically reflect the stages in the development of the log analysis tool using Machine Learning techniques and integration with language models. The organization of the chapters makes for fluid reading, from the initial contextualization to the presentation of the results and conclusions. The introduction sets out the background to the project, the main objectives and the motivation for its development. It is followed by the State of the Art chapter that

covers the concepts, methodologies and technologies that provide the theoretical basis for the work, such as classification algorithms, clustering and RAG techniques.

The Methodologies, Technologies and Tools section details the methods adopted, the libraries used and the architectural decisions. The Activities Developed section describes the practical implementation of the solution, including data processing, training and application of ML models, integration with LLMs and the graphical interface built with FastAPI and Streamlit. The results are also presented, with metrics, graphs, and example outputs. Finally, the Conclusion summarizes the contributions of the project, the difficulties faced and proposes future improvements, such as the evolution of integration with LLMs and the scalability of the solution.

2 State of the Art

This chapter presents the theoretical and conceptual framework of the project, exploring the main concepts, methodologies and technologies involved in log analysis using ML in the Automotive Industry. Existing approaches will be analyzed, as well as the main sources of information consulted to support the development of the tool.

2.1 System Feedback (Logs)

Log files are automatically generated records that include detailed information on events, activities, and operations that take place within software programs, operating systems, or network settings. Time-stamp information including user actions, system problems, application requests, security events, and performance indicators are all recorded in these files. Logs are crucial for diagnosing issues, detecting security breaches, keeping an eye on system health, and guaranteeing compliance. Depending on the system design, they can be kept locally or centrally and exist in a variety of formats. Logs are essential to observability in contemporary systems, and they are frequently examined using automated tools or machine learning approaches to find trends and abnormalities [2].

2.2 Log Analysis

Log analysis is an essential process in monitoring and debugging errors, allowing you to identify patterns, anomalies, and events. Logs are automatic records generated by computer systems that document events, errors, and interactions. In the Automotive

Industry, log analysis is used to look for failures, optimize processes and ensure the correct functionality and safety of the system.

This analysis plays a key role in fault detection, defect prevention and process optimization in the Automotive Industry [3]. Automatic monitoring systems allow you to identify abnormal patterns and improve efficiency in production management.

Traditional log analysis methods rely on manual inspection and predefined rules. However, these approaches have limitations in terms of scalability and efficiency, leading to the need for automated machine learning-based solutions.

2.3 Artificial Intelligence (AI)

Artificial Intelligence is the branch of computational science dedicated to the creation of systems with the ability to perform tasks such as reasoning, learning, planning and creativity, associated with intelligent beings, such as humans. These tasks include problem-solving, pattern recognition, natural language understanding, decision-making, and learning based on experience.

AI evolved a lot due to the great advances in computing capacity, due to the large amount of data available and the development of more sophisticated algorithms [4]. This field encompasses many other subdomains, such as Data Science, Machine Learning, Large Language Models, in order to complete the AI ecosystem.

In practical terms, many AI tools are implemented through machine learning techniques, allowing you to make predictions and identify patterns in large amounts of data. Deep learning also stands out here, a subfield that uses artificial neural networks with multiple layers to extract more complex representations from data. In tasks such as computer vision, speech recognition, and natural language processing, this approach has been very effective. A great example of the power of deep learning are LLMs, which use it to train with huge amounts of text, to generate coherent responses, perform advanced reasoning tasks, and understand natural language [4].

Several models such as GPT or LLama present a major advance in the way AI interacts with humans. In addition, Data Science supports with techniques and resources for the treatment, evaluation, and reading of the data used in these models, being one of the essential components of the AI ecosystem. These themes will be explored in greater depth in the following chapters.

2.4 Data Science

Data Science is the study of data, with the help of tools, techniques, and creativity, such as mathematics and computer science, to gain insights into business decisions to discover patterns and information hidden in data. This area combines programming, statistics, domain knowledge, and visualization to transform large volumes [5].

Data science is important because of its ability to revolutionize the way organizations operate, make decisions, and relate to the customer [6]. It has an impact on various sectors, from healthcare with AI-assisted diagnostics, the automotive industry, with the detection of patterns in tests, and others such as finance, education, logistics, and public administration [7].

2.5 Machine Learning

Machine learning (ML) is a broad field of AI that focuses on developing algorithms and models that can learn from data to make predictions or decisions. ML encompasses various methodologies, each tailored to specific data structures and learning paradigms. [8] The primary approaches include **supervised**, **unsupervised**, **semi-supervised**, and **reinforcement learning**.

Each of these learning paradigms contributes uniquely to the advancement of machine learning, offering diverse tools and techniques to address a wide array of real-world problems.

2.5.1 Supervised Learning

Supervised learning is a machine learning technique that uses labeled datasets to train models that can predict outcomes or classify data based on new inputs. Each workout includes features and a label, allowing the model to accurately map inputs to outputs [9].

In the course of training, the model optimizes its internal parameters to minimize the difference between predictions and actual results, using appropriate loss functions. This is an iterative process that continues until the model achieves satisfactory performance, and is then evaluated with test data to be able to assess its generalizability.

This approach is widely used in two main problems:

- **Classification**: The goal is to be able to assign entries to discrete categories. Some examples include spam detection in emails or recognition of handwritten digits. Common algorithms are Support Vector Machines, Random Forest, Decision Trees, K-Neatest Neighbors [10].
- **Regression**: it serves to predict continuous values, such as the price of a house or the temperature for the next few days. Some of the common algorithms are Linear Regression, Ridge Regression, and Lasso Regression [10].

This supervised learning is widely used in various areas, such as

- **Fraud detection**: identification of suspicious financial transactions [11].
- **Image Recognition**: Classification of Objects in Digital Images [12].
- **Predictive analytics**: Predicting market trends or consumer behavior [13].

It has some advantages such as having very accurate results when trained on quality data, and it has the ability to generalize new data similar to that of training [14]. On the other hand, it needs large amounts of labelled data, which can be costly and time-consuming. And there is a risk of overfitting (when the algorithm adapts too much to the training data), if the model is too complex for the available data set.

2.5.2 Unsupervised Learning

Unsupervised Learning is an ML technique that enables interpretation analysis of datasets without predefined labels. Unlike supervised learning, which needs labeled data for models to be trained, unsupervised learning identifies patterns, structures, and relationships in the data autonomously, without human intervention [15].

In this approach, algorithms analyze the data to discover natural groupings, associations, or latent structures. It aims to understand the underlying organization of the data, allowing insights that would not be noticeable with traditional methods. Some of the most common techniques contain:

Clustering: Groups similar data based on common characteristics. Some example algorithms are K-Means and DBSCAN [16].

Dimensionality Reduction: simplifies complex data sets while preserving the most relevant characteristics [17]. Some of the most used techniques are Principal Component Analysis (PCA) and Singular Value Decomposition (SVD) [18].

This unsupervised methodology is widely used in several areas:

Customer segmentation: identifies groups of customers with similar behaviors or preferences, assisting in personalized marketing strategy [19].

Anomaly grouping: identifies atypical patterns or behaviors in data, very useful in predictive maintenance and detection of fraud or errors, and in cybersecurity [19].

Computer vision: In tasks such as object recognition, this methodology helps identify common visual features rather than the need for labeled data [19].

Medical imaging: helps in the detection and segmentation of medical images, such as X-rays and MRIs, improving diagnoses and treatments [19].

It has some advantages such as the ability to work with large amounts of unlabeled data, and allows you to discover hidden patterns that are sometimes not evident in traditional analytics [20]. On the other hand, interpreting the data can be challenging, requiring constant validation. And there is a risk of identifying irrelevant patterns or coincidences with no practical meaning.

2.5.3 Semi-Supervised Learning

Semi-Supervised Learning is an ML technique that combines both elements of supervised and unsupervised learning, using both labeled and unlabeled data simultaneously to train models on regression and classification tasks. This is a very useful approach in scenarios where obtaining large volumes of labeled data is time-consuming and costly, but there is also an abundance of unlabeled data available [21].

Throughout the training, this technique uses a set of data that is labeled in order to guide the model, while also exploring the unlabeled data to identify underlying structures and patterns. This combination allows the model to improve its accuracy, even if the labeled data is few [22].

This methodology is widely used in several areas, namely those already mentioned above in the supervised and unsupervised models, since it encompasses both learnings [23].

It also has advantages in reducing the need for labeled data, which allows reducing costs and preparation times, and also improves the accuracy of the model by obtaining insights from unlabeled data. In contrast, its effectiveness depends on

the quality of the unlabelled data, and there is a risk of introducing noise if the unlabelled data is of low quality or unrepresentative [24].

2.5.4 Reinforcement Learning

Reinforcement Learning involves an agent learning to make decisions by interacting with an environment, receiving feedback in the form of rewards or penalties [25]. This trial-and-error approach enables the agent to learn optimal behaviors over time. Applications include robotics, where agents learn to navigate or manipulate objects, and game AI, where agents develop strategies to maximize scores.

This learning method can be divided into three components, which in this case are, the agent, who is the one that makes the decisions, the environment, where the agent makes his interactions, and the reward, which serves to guide the agent's behavior if it was right or wrong [26]. During the process, the agent observes the state of the environment at the moment, chooses an action based on a decision strategy, puts that action into practice, receives a reward, and observes the new state [27]. Over time, the agent adjusts his policy to choose action to choose the action that favor him in rewards [28].

Reinforcement Learning is very useful in several areas of our daily lives, such as robotics, where agents learn to perform tasks, in games, where agents learn the best strategies, in autonomous vehicles, where they make decisions in real time in the environment where they are, and in many other areas [29].

2.5.5 Deep Learning

Deep learning is a subdivision of ML that is based on multi-layered neural networks that are known as deep neural networks. The power of these architectures stands out mainly in the modeling of complex patterns and representations, being notorious in high-dimensional data, such as images, audio and natural language

[30]. Deep Learning has allowed many advances in certain areas such as computer vision (facial recognition), natural language processing (translation, summarization) and time series analysis [30]. The most common models include Convolutional Neural Networks (CNN) and/or Recurrent Neural Networks (RNN), which are instrumental in powering LLMs such as LLama and GPT [31].

2.5.6 Machine Learning in Log Analysis field

Integrating ML into log analysis aims to significantly enhanced the efficiency and accuracy of monitoring and diagnosing complex software systems. Traditional log analysis methods often struggle with the vast volumes of data generated by modern applications, making manual inspection a very hard process in terms of time and effort spent. ML techniques automate the processing of log data, enabling real-time detection of anomalies, prediction of system failures, and extraction of actionable insights [32].

Applications of Machine Learning in Log Analysis:

- **Anomaly Detection:** ML models can identify deviations from normal patterns in log data, signaling potential system issues or security threats. For instance, deep learning approaches have been employed to detect anomalies in log data, capturing patterns and reporting unexpected log events without prior modeling of anomalous scenarios [33].
- **Event Classification and Clustering:** By categorizing log entries and grouping similar events, ML facilitates the identification of trends and correlations within the data. Techniques such as clustering have been proposed for anomaly detection in log files, aiding in the organization and analysis of large datasets [34].
- **Predictive Analysis:** ML algorithms can forecast potential system failures by analyzing historical log data, allowing for proactive maintenance and reducing downtime. This predictive capability is crucial for maintaining system reliability and performance [34].

Advantages of Machine Learning in Log Analysis:

- **Enhanced Efficiency:** Automating log analysis with ML reduces the need for manual intervention, allowing engineers to focus on strategic tasks. This automation accelerates the identification and resolution of issues within the system [35].
- **Improved Accuracy:** ML models can process extensive log data with high precision, minimizing human error and increasing the reliability of insights derived from log analysis [35].
- **Proactive Issue Resolution:** By detecting anomalies and predicting failures early, ML enables organizations to address problems before they escalate, ensuring system stability and reducing the risk of significant disruptions [35].

Incorporating machine learning into log analysis represents a significant advancement, offering deeper insights and more rapid responses in managing complex software systems [36].

For the context of this work, several ML techniques were applied in order to classify and analyze the registration data, including traditional models such as Random Forest and DBSCAN and later Deep Learning models such as RNNs to replace Random Forest, to be able to process natural language. These models were trained and used to detect and group error patterns in automotive system logs, to show how ML can be effectively applied to real-world problems, especially in industry.

2.6 Generative AI

Generative AI refers to a class of artificial intelligence models capable of generating new content, such as image, audio, code, or text, based on patterns learned from large volumes of data [64]. These generative models can create new and coherent information with the input data.

These models have become widely known with the introduction of generative language models such as GPT [65] or BERT and more recently with models optimized

for local inference such as Gemma and LLaMA [66]. Many of these models are based on transformers, an architecture that has revolutionized the way text is processed by AI [67].

In the context of this project, Generative AI was essential to enable semantic interaction with the log files. After data preparation and classification, it was possible to apply an LLM fed with vectorized chunks of the logs to answer user questions in a natural way. This capability was made possible by the generative component of the model, allowing informative and relevant responses to be produced from the data.

In addition, Generative AI was used to answer technical questions about the data, allowing the developed tool to serve as an intelligent assistant in the log analysis process, something particularly useful in the automotive industry, where the complexity and volume of data is high.

2.7 Large Language Model (LLM)

Large Language Models (LLMs) are a breakthrough in artificial intelligence [37]. Models like these employ deep learning techniques, especially transformer architectures—to process and generate human-like text. By being trained on vast amounts of unlabeled data, LLMs learn complex patterns, enabling them to perform diverse tasks such as translation, summarization, and code generation. The evolution of LLMs has revolutionized the field of Natural Language Processing (NLP) by offering unprecedented scalability and versatility [38].

2.7.1 How LLMs Work

LLMs operate using neural networks that contain billions of parameters to process and understand human languages. During training, LLMs are exposed to massive datasets where they adjust their internal weights to minimize prediction errors, thereby learning structure, syntax, and semantics of the language. The architecture of these models is based on transformers, which process entire sequences of text

simultaneously rather sequentially. This approach, particularly through mechanisms like self-attention, enables models to determine the contextual importance of each word relative to others in a sentence. Additionally, LLMs incorporate context, and generating coherent outputs [39].

2.7.2 Applications of LLMs

The capabilities of LLMs have led to their widespread adoption across various sectors. In customer service, they power chatbots and virtual assistants that can engage users in dynamic and natural conversations. In content creation, these models are used to generate articles, creative writing, and even assist in programing tasks by generating code snippets based on natural language instructions [40]. Beyond these, LLMs play a crucial role in language translation and text summarization, making vast amounts of information more accessible. Their adaptability also finds use in educational tools, where they help tailor learning experiences by providing personalized study materials and answering questions, consequently transforming the way people interact with information [40].

2.7.3 Challenges and Future Directions

Despite their impressive performance, LLMs confront multiple critical limitations. Their development requires enormous computational power and economics investments, leading to issues about energy usage and environmental consequences [41]. Trained on vast internet-sourced datasets, these systems risk absorbing and reproducing societal biases embedded in the material, potentially perpetuating harmful stereotypes or factual errors [42]. Another challenge is the diminishing returns in performance gains when model's era scaled up indefinitely, prompting researchers to explore more efficient methods. Future advancements will likely concentrate on strengthening logical processing and situational awareness while minimizing resource expenditure and addressing ethical concerns. Innovations in architecture and training techniques are expected to play a pivotal role in shaping

the next generation of LLMs, aiming to develop more reliable, principled, and sustainable artificial intelligence systems [43].

2.8 Encoding

Encoding is the process of converting data from one form to another to make it suitable for computational processing. In the context of machine learning and data analysis, encoding typically involves transforming categorical, numerical, or textual data into numerical formats that algorithms can interpret.

There are different types of encoding depending on the nature of the data:

- **Categorical data** often requires techniques like Label Encoding, which assigns unique numerical values to each category, or One-Hot Encoding, which creates binary vectors for each category. Other methods such as Binary Encoding, Hash Encoding, or Target Encoding are used depending on the data complexity and the algorithm's requirements.
- **Numerical data** may be transformed using techniques such as Z-score normalization (Standardization), which centers values around the mean with a standard deviation of one, or Min-Max Scaling, which rescales features to a fixed range, often [0, 1]. Log transformation and binning can also be applied to correct skewed distributions or reduce the influence of outliers.
- **Textual data** requires conversion into numerical representations. Bag of Words (BoW) and TF-IDF (Term Frequency-Inverse Document Frequency) are widely used for sparse vector representations. More advanced approaches like Word Embeddings (e.g., Word2Vec or GloVe) allow capturing semantic meaning in dense vector spaces.

Encoding is a foundational step in the data preprocessing pipeline, as it ensures that all features are in a compatible format and scale, allowing machine learning models to learn effectively without being biased by differing data types or magnitudes.

2.8.1 Encoding Techniques Used

In the develop toll, the following encoding techniques were implemented:

- **TF-IDF Vectorization**: To process the textual data in the logs, the TfidfVectorizer tool was used. This method converts raw text into numerical vectors by considering both the frequency of a word in a document and its inverse frequency across all documents. This helps emphasize relevant terms while downplaying common or non-informative ones.
- **Standardization with StandardScaler**: After vectorization, StandardScaler was applied to normalize the TF-IDF vectors. This ensures that each feature has a mean of zero and a standard deviation of one, promoting balanced learning in the machine learning model and avoiding bias from features with larger magnitudes.

These encoding steps were essential to transform the raw log data into a structured and numerically consistent format, ensuring compatibility with the Random Forest classifier and subsequent clustering with DBSCAN, that will be discussed further.

2.9 Software Development Methodologies

Traditional software development methodologies refer to approaches and practices that have been established over time for developing a product. They are based on structured and sequential processes with the intention of achieving effective and high-quality results. They provide defined guidelines and steps, from design to final delivery.

There are several traditional software development methodologies, each with its own distinct characteristics and processes. Some of the most popular examples include the Waterfall Model, V-Model, Rational Software Development, and the Spiral Model. These methodologies have been widely adopted in the technology industry and have been successfully used in various software development processes.

One of the main characteristics of traditional development methodologies is their sequential approach, where the steps of the process are executed in a fixed, linear order, where each phase must be completed before moving on to the next. They also

require comprehensive documentation at all stages, such as requirements specifications, technical design, test reports, and user manuals, making it easier to use and maintain software in the future.

3 Methodologies, Technologies and Tools Used

The development of the log analysis tool followed an iterative approach according to the agile methodology and the pyramid scheme, which allowed for continuous adjustments throughout the process. This methodology involved a structured pipeline for ML starting with the exploration and cleaning of the data, followed by feature engineering, selection and training of the models, and finally evaluation and validation to ensure the robustness of the results.

3.1 Methodologies

3.1.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a technique that enhances the response of LLM, so it references an authoritative knowledge base outside of its training data sources before generating a response. While LLMs are trained on extensive datasets and use billions of parameters to perform tasks such as answering questions, translating or completing tests, RAG improves their accuracy and relevance by grounding their answers in up-to-date or domain-specific information. It is a cost-effective approach that enables organizations to adapt LLM outputs to their own data without the need for retraining, offering a practical way to ensure responses are accurate, relevant and context aware [44].

3.1.2 Clustering

Clustering is an unsupervised ML technique that classifies and organizes different data points or objects, considering their similarity or pattern. It helps to discover hidden structures of groups in data when you don't have predefined labels or categories. It was used to identify natural groupings in the data, which can provide insights, improve segmentation, or support decision-making [45].

3.1.3 Classification

Classification is a type of supervised learning ML algorithm that assigns labels or categories to input data based on patterns it has learned from a labeled dataset. It can be used to determine whether an email is a spam or not, or to identify whether an image contains a cat or a dog. It was used to automatically categorize the data into predefined classes, helping automate decisions and detect patterns in structured information [46].

3.1.4 MVC Architecture

MVC is a software design pattern that divides the application into three layers connected to each other. It facilitates the separation of responsibilities, divides business logic, presentation, and control of application flow. This structure allows for cleaner, more modular and easy-to-maintain code.

The three main components are:

Model: Represents the business logic and manipulation of data. Interacts with the database and provides the data to the View.

View: It is the layer that presents the visual part of the data to the user. Shows information dynamically and iteratively.

Controller: Responsible for being the intermediary between the Model and the View. Processes inputs for the user, interacts with the Model for data retrieval, and keeps the View up to date.

3.2 Tools, Technologies and Libraries

3.2.1 Facebook AI Similarity Search (FAISS)

FAISS stands for Facebook AI Similarity Search and is an open-source library developed to make it easy to search for similar vectors and group dense vectors. This tool allows you to create indexes and perform searches quickly and efficiently in terms of memory usage.

One of the biggest highlights of FAISS is the ability to use the CPU to speed up different indexing methods, significantly improving search performance [47].

3.2.2 Hugging Face Transformers

Hugging Face Transformers is an open-source library that provides a wide variety of pre-trained models for natural language processing (NPL), such as BERT, GPT, and T5. It supports tasks like sentiment analysis, text classification, translation, question answering, and more. It was used to easily integrate powerful NLP models into the application without the need to train them from scratch [48].

3.2.3 Pandas

Pandas is a Python library designed for data manipulation and analysis. It introduces two main data structures, Series and Dataframe, which make it easy to handle and explore structured datasets efficiently. It was used to clean, transform, and analyze data in a tabular format with readable and concise code [49].

3.2.4 NumPy

NumPy is a foundational Python library for numerical computing, providing support for large multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them. It was used in the project for fast mathematical operations on arrays and as the base layer for many other scientific computing libraries in Python [50].

3.2.5 Matplotlib

This technology is a data visualization library in Python that enables the creation of static, animated, and interactive plots. It is often used for production line graphs, bar charts, histograms, and scatter plots [51]. In the project it was essential to visually explore data and present insights through charts and graphs.

3.2.6 PostgreSQL

PostegreSQL is a powerful, open-source relational database management system known to its reliability, feature richness, and standards compliance. It supports complex queries, indexing, transactions, and a wide range of data types [52]. It was used to store and manage embeddings from the data that was processed to feed the LLM.

3.2.7 Python

Python is a high-level, multi-paradigm, cross-platform programming language, recognized for having a clear syntax and exceptional readability. It is designed to develop developer productivity, as it supports object-oriented, functional and procedural programming, adapting to diverse development contexts. Its large library offers ready-made modules for tasks such as file manipulation, network operations, and data processing, reducing the need for additional code.
This language excels in areas such as task and process automation, data analysis, artificial intelligence, and web development, due to frameworks such as Flask and Django, as well as specialized libraries such as Pandas, NumPy, and TensorFlow. It is also used in DevOps scripts, system integration and rapid prototyping, favoring agile development cycles, making it versatile and in-demand skill in the modern technology [53].

3.2.8 Visual Studio Code

Visual Studio Code is a free and lightweight yet highly capable code editor that works across all desktop operating systems. It comes with built-in support for JavaScript,

TypeScript and Node.js and has a rich ecosystem of extensions for other programming languages like C++, C#, Java, PHP, Python and Go, runtimes such as .NET and Unity, environments such as Docker and Kubernetes, and clouds like Amazon Web Services, Microsoft Azure, and Google Cloud Platform [54].

3.2.9 Jupyter Notebook

Jupyter Notebook App is a server-client application that allows editing and running notebook documents via web browser or on a local desktop. In addition to displaying/editing/running notebook documents, the Jupyter Notebook App has a dashboard, a control panel showing local files and allowing to open notebook documents or shutting down their kernels [55].

3.2.10 PlantUML

PlantUML is an open-source tool created in 2009 by a French developer named Arnaud Roque. It permits the creation of technical diagrams using a dedicated, intuitive language, and it is available as a plugin in many IDEs. It has many advantages compared to drawing diagrams, because code is easier to maintain than drawn diagrams, PlantUML draws and formats the diagram from the code, there are numerous plug-ins exist in documentation tools and IDEs to generate diagrams [56].

3.2.11 BitBucket

Bitbucket Cloud serves as a Git-powered hub for code hosting developed by Atlassian, built for teams and software projects. [57] It provides a comprehensive environment for managing code, collaborating with team members, automation workflows, and integrating with other development tools like Jira and Trello, making project tracking and code management efficient. BitBucket is available as a cloud service for larger organizations.

3.2.12 Podman Compose

Podman is a container management tool that offers a lightweight and daemonless alternative to Docker, allowing users to manage containers without the need for a central daemon process. [58] Complementing Podman is Podman Compose, a tool designed to simplify the orchestration of multi-container applications.

Podman Compose is an extension of Podman that provides functionality like Docker Compose. Docker Compose simplifies the definition and orchestration of multi-container applications, allowing the users to define a multi-container application in a singles file and then spin up the entire application stack with a single command.

This tool leverages a YAML-formatted Compose file to define the services, networks, and volumes of a multi-container application.

How Podman Compose Works:

- It is necessary to create a docker-compose.yml file to define the services, networks, and volumes of the application.
- The users need to define the image, environment variables, ports, and other configuration options for each service, like in the code below.

```
version: "3.8"
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    ports:
      - "11434:11434"
    networks:
      - ml_network
  postgres:
```

```
    image: pgvector/pgvector:pg17
    container_name: pgvector-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Ricardo04
      POSTGRES_DB: embeddings
      POSTGRES_HOST: localhost
      POSTGRES_PORT: 5432
    ports:
      - "5432:5432"
```

- The Compose file may include definitions for custom networks and volumes, allowing services to communicate and share data, as exemplified in the code below.

```
networks:
      - ml_network

volumes:
  pg_data:

networks:
  ml_network:
    driver: bridge
```

- Users use Podman Compose commands to manage the multi-container application. Commands include podman-compose up to start the application and podman-compose down to stop and delete containers.

3.2.13 FastAPI

API stands for Application Programming Interface. An API is a software layer that enables two applications to communicate with each other.

FastAPI is a modern and fast web framework for building APIs with Python based on standard Python type hints. It helps developers build applications quickly and efficiently [59].

3.2.14 Streamlit

Streamlit is an open-source app framework designed to help developers and data scientists build interactive, data-rich web applications quickly and easily. Unlike traditional web development frameworks that require extensive knowledge of HTML, CSS, or JavaScript, Streamlit enables users to create web apps using Python scripts. This significantly lowers the barrier for creating and sharing data applications, dashboards, and machine learning tools [60].

3.2.15 Sckikit-learn

Sckit-learn is an open-source library in Python, specially designed for Machine Learning. It contains simple and efficient tools such as classification, regression, clustering, and dimensionality reduction algorithms [61]. It is reused in different situations because it offers a user-friendly interface and reliable implementations of many algorithms, making it accessible for everyone. It was used to quickly and easily apply proven ML algorithms to the data without having to implement them from scratch.

3.2.16 LangChain

LangChain was used as a tool and technology to support the integration of language models with external data sources, allowing the construction of a RAG system. It is a framework designed to facilitate the development of applications based on LLMs, such as intelligent agents, data assistants or question and answer systems.

This tool allows you to easily connect LLMs to documents, databases or APIs, through mechanisms such as semantic indexing, prompt chaining and conversational memory management. According to AWS [62], LangChain provides abstractions that help programmers combine language models with other sources of knowledge and programmatic logic, making it ideal for building systems that answer questions based on dynamic data.

The use of LangChain in this project was fundamental to orchestrate the flow between the local model, the indexed data and the generated answers, enabling an interface of questions about the processed data.

3.2.17 TensorFlow

TensorFlow is an open-source tool developed by Google, widely used for building and training ML models. According to the official Google OpenSource description [63], TensorFlow provides an extensible platform for numerical computation with support for deep neural networks, making them one of the most adopted libraries in the field of artificial intelligence.

In this project, TensorFlow was used specifically to train an LSTM network, with the aim of analyzing text strings in the column that contains the most relevant information from the logs. Its integration with Keras made it easy to build the model in a modular and efficient way, with support for tokenization, embedding, and LSTM layers, as well as features for training, validation, and persistence of the model.

3.2.18 Keras

Keras is a high-level API for building and training Deep Learning models, developed with the goal of being easy to use, modular and extensible. It is currently integrated into TensorFlow as the main interface for creating neural networks. Its intuitive and object-oriented syntax allows you to define models quickly and understandably, which speeds up the prototyping and experimentation process.

In the project in question, Keras via Tensorflow.keras was used to create and train an LSTM model, a variant of recurrent neural networks, with the aim of analyzing text sequences and detecting patterns associated with errors. Using Keras allowed us to define the model declaratively, adding layers such as Embedding, LSTM, Dropout and Dense in a simple and clear way.

In addition, Keras facilitates the process of compiling and training the model, as well as managing weights, metrics and integration with visualization and saving mechanisms of the trained model.

4 Developed Activities

This chapter provides context about the project, consistent in a tool and a brief overview of the assigned internship project. The tool aims to facilitate the work of individuals in tester roles by reducing the time spent analyzing logs. It was designed to assist in the automatic identification, classification, and correlation of events extracted from log files generated during software or system testing processes.

The project involved the application of machine learning techniques, namely supervised models for error detection (e.g., Random Forest), unsupervised models for pattern discovery (e.g., DBSCAN) and deep learning, as well as the use of vectorization methods to extract relevant features from textual log data. These models were trained and reused through a structured pipeline, with intermediate steps and outputs saved for reproducibility and future analysis.

To support the scalability and modularity of the solution, all components were containerized using Podman, including the backend, frontend, database, and language model (LLM). The system includes a Retrieval-Augmented Generation (RAG) architecture, where embeddings of processed log files are generated, stored in a vector database, and later queried using a local LLM. This enables users to ask natural language questions about the data and receive contextualized answers based on the indexed embeddings, improving accessibility and interpretability for both technical and non-technical stakeholders.

The remaining of this chapter presents the research, the development, and the obtained results.

4.1 Research

The project started with a comprehensive learning phase aimed at building a robust foundation in machine learning (ML) and Python programming. This phase was crucial for equipping myself with the necessary skills and knowledge to effectively explore and evaluate various ML algorithms.

Online Courses and Skill Acquisition

I began by enrolling in online courses offered by reputable platforms such as Google and LinkedIn. These courses provided structured and in-depth coverage of machine learning concepts and Python programming techniques. Through these courses, I gained proficiency in:

- Understanding fundamental ML concepts, including supervised and unsupervised learning, model evaluation, and performance metrics.
- Implementing ML algorithms using Python libraries such as scikit-learn and TensorFlow.
- Preprocessing data, feature engineering, and handling datasets to prepare them for analysis.

Independent Research

Complementing the structured courses, I conducted independent research to deepen my understanding of machine learning algorithms and their practical applications. This involved:

- Exploring academic papers, articles, and reputable online resources to gather diverse perspectives on ML methodologies.
- Investigating the theoretical underpinnings of various algorithms, including their mathematical foundations and use cases.
- Staying updated with the latest advancements and trends in the ML field by following industry blogs and forums.

Exploration and Evaluation of Machine Learning Algorithms

With a solid theoretical background, I proceeded to explore and evaluate different types of machine learning algorithms to determine their suitability for various tasks. The process involved:

- **Algorithm Selection:** Identifying a range of algorithms to study, including:
    - **Supervised Learning Algorithms:** Linear Regression, Decision Trees, Support Vector Machines (SVM), and Neural Networks.
    - **Unsupervised Learning Algorithms:** K-Means and DBSCAN Clustering, Hierarchical Clustering, and Principal Component Analysis (PCA).
    - **Semi-Supervised Learning Algorithms:** Self-training and Co-training methods.
- **Evaluation Criteria:** Establishing metrics to assess the performance of each algorithm, such as accuracy, precision, recall, F1-score, and computational efficiency.
- **Practical Implementation:** Applying these algorithms to benchmark datasets to observe their behavior and performance in real-world scenarios. This hands-on experimentation allowed me to:
    - Gain insights into the strengths and limitations of each algorithm.
    - Understand the impact of parameter tuning and feature selection on model performance.
    - Develop the ability to choose appropriate algorithms based on the specific characteristics of a given dataset and problem domain.

Findings and Insights

Through this investigation process, I discovered that the effectiveness of an ML algorithm is highly contingent on the nature of the data and the problem at hand. For instance:

- Linear models like Linear Regression perform well on datasets with linear relationships but may falter with complex, non-linear data.
- Tree-based methods such as Decision Trees and Random Forests offer interpretability and handle categorical data effectively but can be prone to overfitting without proper pruning.
- Neural Networks, while powerful for capturing intricate patterns, require substantial data and computational resources, and are often considered "black boxes" due to their lack of interpretability.

This exploration underscored the importance of understanding the underlying assumptions and mechanics of each algorithm to make informed decisions tailored to specific analytical objectives.

4.2 Modeling

The modelling phase is a fundamental part of the creation cycle of any system or project, as it allows to approach, in an abstract and systematic way, the fundamental elements of the project, their characteristics and the relationship between the different elements. The objective of this phase is to ensure a clear and shared understanding of the system to be developed, making its implementation and future maintenance simpler.

In this section, the main functional and non-functional requirements are presented, as well as the modelling diagrams that support the design of the system. These include, for example, use case diagrams, sequence diagrams, component diagrams, among others. Modelling makes it possible to structure the technical decisions made and anticipate possible challenges in implementation.

By following a modelling-driven approach, it is possible to align the objectives of the system with the needs of users and technical constraints, promoting a coherent, scalable and effective solution.

4.2.1 Functional Requirements

In this section, the functional requirements of the system are presented:

RF1: The user inserts a file to be trained.

RF2: The user sends a file to be analyzed by LLM.

RF3: The user asks questions to LLM.

RF4: The application should store trained models, training data, and inference logs.

RF5: The application must apply a trained model with semi-supervised learning to classify the input.

RF6: The application should save the input already trained.

RF7: The application must allow you to receive a file.

RF8: The application must split and save the file in embeddings.

RF9: The application's LLM must be able to read/parse the uploaded file.

RF10: LLM should answer questions based on the file.

4.2.2 Non-Functional Requirements

In this subchapter, the non-functional requirements of the system are presented:

RNF1: The UI should feature a user-friendly interface.

RNF2: The system must be available when needed.

RNF3: The system should be able to run on different platforms.

RNF4: The system should be easy to maintain and update.

RNF5: Be able to handle the required number of users.

RNF6: The maximum acceptable time for LLM to generate a response to a question about the processed file should not exceed 2 minutes.

RNF7: It should be possible to scale computational resources (CPU, GPU, memory) as needed.

RNF8: The system should provide feedback on the status of the file processing (e.g. "File received", "Training in progress", "Embeddings generated", "Ready for questions").

RNF9: Files uploaded by users should be stored securely.

RNF10: Monitoring mechanisms should be implemented to detect and alert on system failures.

4.2.3 Use Case Diagram

This section introduces the Use Case Diagram, which outlines the primary interactions between the users and the system. The goal is to clarify the functionality of the user's perspectives by identifying the key actions supported by the platform.

In the context of this project, the system was designed to enable a user to:

- Upload log files for analysis.
- Automatically detect and classify errors using ML models.
- Explore clusters of similar log events using DBSCAN.
- Interact with a local LLM in natural language to ask questions about the processed data using a RAG architecture.
- Visualize the results and download processed outputs.

The diagram presented in Figure 5 emphasizes the roles, and the main functionalities provided through the Streamlit frontend, backed by a modular architecture that handles data, models, and natural language responses.

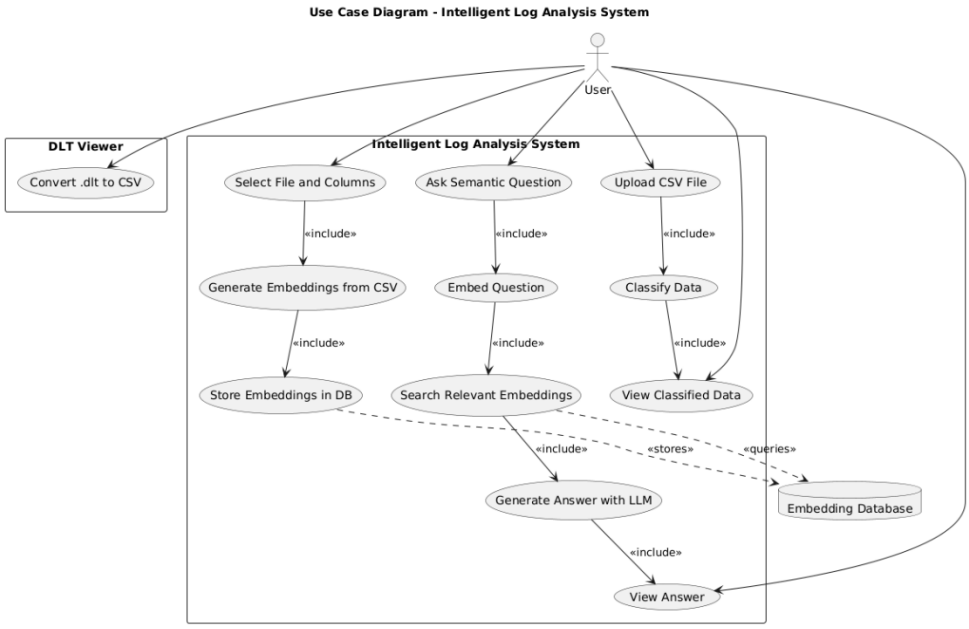

Figure 2 - Use Case Diagram

4.2.4 Component Diagram

This section presents the component diagram, which represents the high-level architectural structure of the developed system. The diagram highlights the main components and their relationships, showcasing how the system was modularized using containerized architecture, Specially, the solution was divided into three core components:

- **Implementation of a backend component**, using FastAPI, responsible for data processing, model inference, and file management.
- **A frontend component**, built with Streamlit, that provides a user-friendly interface for uploading files, visualizing results, and querying the system.
- **A third container running a local LLM** integrated via Ollama, used in a RAG setup to allow natural language interaction with the processed log data.

The diagram presented in the Figure 6 shows how a modular structure promotes maintainability, scalability, and separation of concerns, ensuring that each part of the system can evolve independently while communicating efficiently through well-defined interfaces.

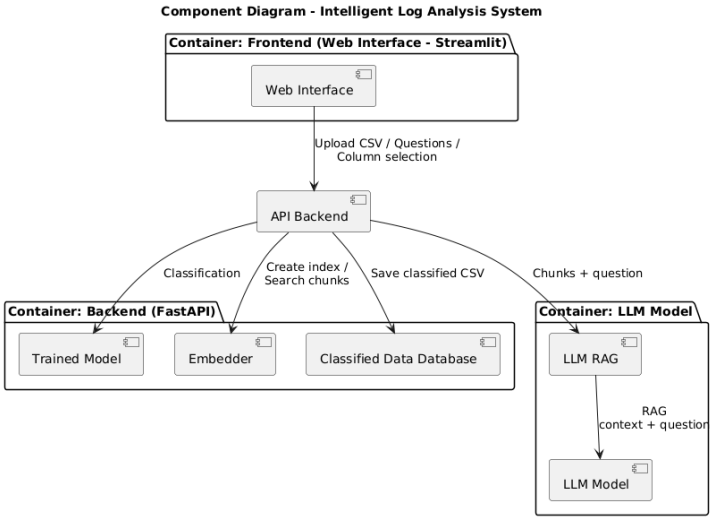

Figure 3 - Components Diagram

4.2.5 Sequence Diagram

This section presents the Sequence Diagram, which describes the dynamic interactions between system components during the processing of a log file.

The typical sequence starts when the user uploads a file through the Streamlit frontend. The request is sent to the FastAPI backend, which triggers a series of actions:

- The file is processed using preprocessing scripts and passed to training models for classification and clustering.
- The results are saved, and an embedding is created for use in RAG-based querying.
- The user can submit questions about the logs, which are passed through the RAG pipeline involving HuggingFaceEmbeddings and the local LLM, returning a natural language response.

The diagram in the Figure 6 helps illustrate the flow of data and control between the user, backend components, and LLM service during a typical usage scenario

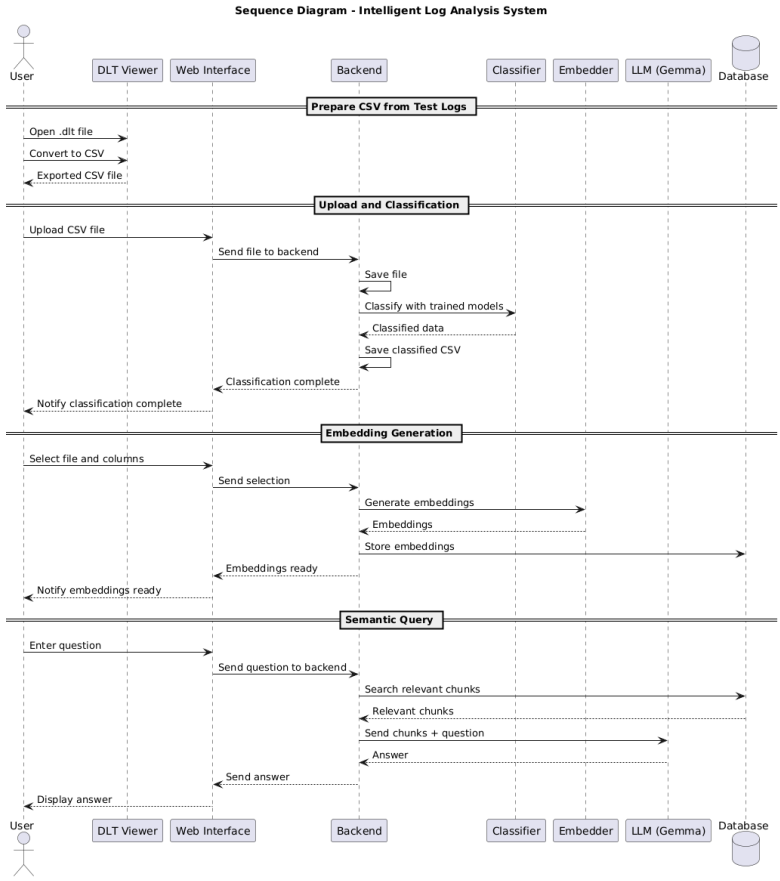

Figure 4 - Sequence Diagram

4.2.6 Activity Diagram

In this section is introduced the Activity Diagram, which models the main workflow of the system, from data input to semantic querying using a local LLM.

The diagram illustrates the system's activities across three main stages:

1. Initial Data Handling:

   The user uploads a log file in CSV format via the Streamlit interface, and the backend is responsible for saving it securely. This is the entry point into the processing pipeline.
2. Classification Phase:

Once the file is saved, it is classified using pre-trained ML models. The output is a new CSV containing the predicted labels and others relevant information, which is then stored for later use.

3. Embedding Generation and Semantic Search:

   The user can choose a previously classified file and define which columns to embed. These embeddings are generated using Sentence Transformers during querying and HuggingFace during indexing. When a user asks a question, the system retrieves the relevant chunk of text based on a vector similarity and forwards both the question and the chunks to a local LLM. The answer is then displayed in natural language through the interface.

The diagram presented in Figure 7 captures the end-to-end flow of the user experience and highlights how the system integrates classic ML techniques with modern RAG capabilities.

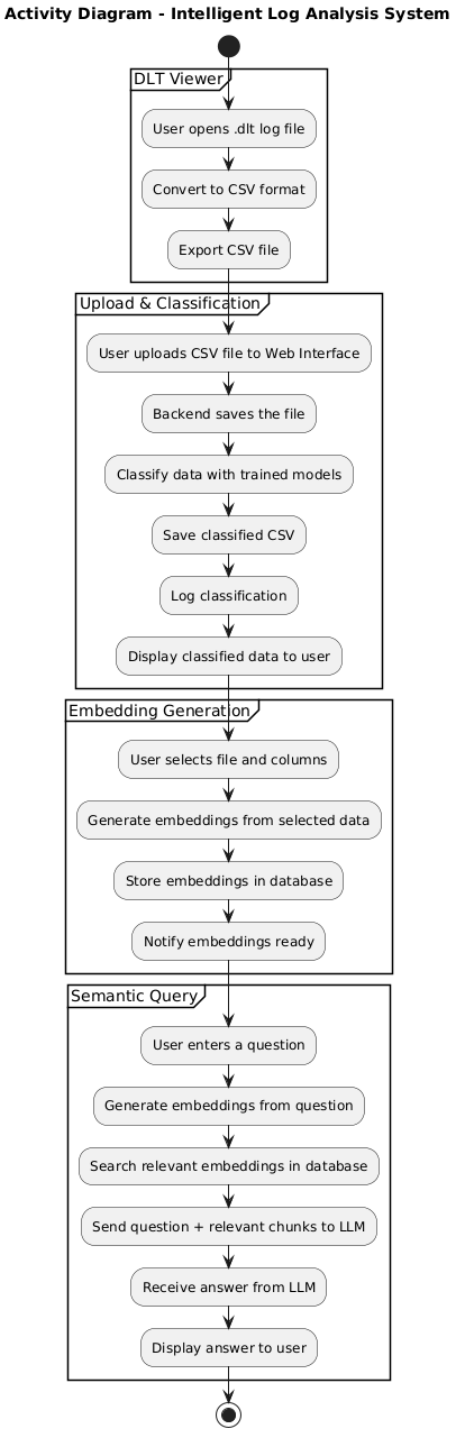

Figure 5 - Activity Diagram

4.3 Development

The development phase of this project followed an iterative and modular approach, starting with the evaluation and testing of different ML models, and progressing towards the implementation of a robust architecture based on the Model-View-Controller design pattern and microservices.

The primary objective was to build an automated system capable of analyzing logs data, detecting patterns and anomalies from the car systems using trained models, and enabling natural language interaction with the processed information through a locally deployed language model. Initially, various ML algorithms, such supervised and unsupervised, were tested to assess their performance in classifying and clustering data extracted from log files. The experiments included models such as Random Forest, XGBoost, KMeans, and DBSCAN. These tests provided valuable insights into the behavior of each algorithm under different data conditions and preprocessing scenarios. Based on the results, Random Forest was selected for its high classification accuracy, while DBSCAN was chosen for its effectiveness in identifying irregular clusters and noise without the need to define the number of clusters in advance.

Following the model selection, the architectural structure of the system was defined. The decision to adopt an MVC and microservices-based architecture was motivated by the need to ensure a clear separation of concerns, facilitate long-term maintainability, and allow for independent scaling of components. Each core component of the system was developed as an independent microservice and containerized using Podman to ensure portability, environmental isolation, and deployment flexibility.

The architecture comprises three main components:

- A **backend service**, implemented with FastAPI, responsible for data processing, model inference, interaction with the database, and API presentation.

- A **frontend interface** development with Streamlit, allowing users to upload CSV files, perform indexing, and interact with the system through a user-friendly interface.
- A **language model (LLM) microservice** is deployed locally using Ollama, and a Gemma model enables users to query the processed data using natural language. This component is enhanced with a Retrieval-Augment Generation (RAG) pipeline supported by HuggingFace, which indexes document embeddings and enables semantic search over the data.

To efficiently store and query the vector embeddings generated from the CSV files, a PostgreSQL database extended with pgvector was integrated into the backend container. This allows for high-performance vector similarity queries, which are essential to support the RAG-based interaction with the LLM.

The following sections describe in detail the process of model training and evaluation, the implementation of each architectural component, the communication between services, and the technologies adopted to ensure scalability, modularity, and efficiency of the system.

4.3.1 Random Forest Training

Random Forest is a supervised learning algorithm based on sets of decision trees. Its operation is based on the creation of multiple trees during the training phase, and the final result is obtained through the aggregation (by majority vote in the classification, or average in the regression) of the individual predictions of each tree. This approach significantly reduces the risk of overfitting that is common in individual decision trees, by introducing variability into the construction process of each tree (through bagging and random selection of subsets).

At the beginning of development, tests were run with Random Forest with labels defined in a csv file filtered by classes that existed in the file data and vectorizing them so that they could be processed in training. But with this cleaning process, the file became too clean and small, and by applying Random Forest's algorithm, the model instead of learning, started to memorize the parameters which led to

overfitting. Overfitting occurs when a model fits too well with the training data (learns a lot from the data), such as a learner memorizing the answers instead of understanding the topic [6], which compromises its ability to generalize to new data. This result indicated the need to reassess the data preparation process and the volume of data available, in order to ensure the robustness of the model and avoid biases in the training process.

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       114
           1       1.00      1.00      1.00       394

    accuracy                           1.00       508
   macro avg       1.00      1.00      1.00       508
weighted avg       1.00      1.00      1.00       508

Accuracy: 1.00
Training Loss: 0.00
Validation Loss: 0.00
```

Figure 6 - Accuracy

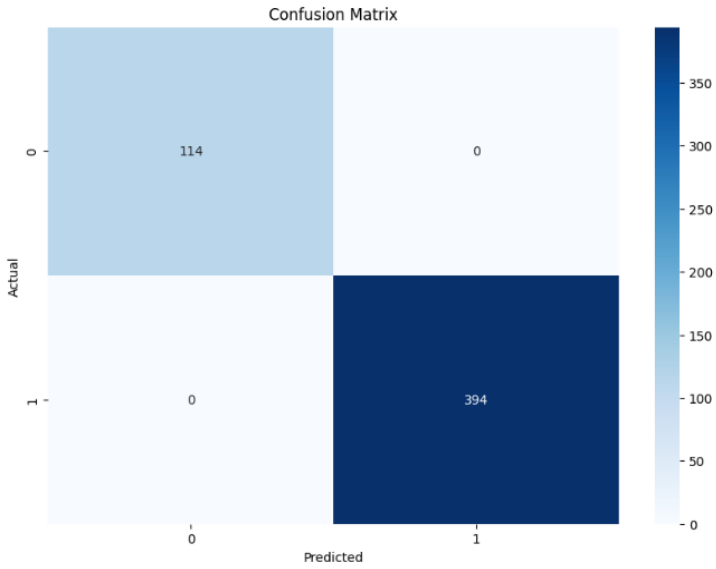

Figure 7 - Confusion Matrix

Then, the same algorithm was tested, but only by cleaning up the rows of the file that were empty or contained null values and vectorizing the column that was to be examined. Graph images extracted from training show stable learning curves and training loss and a confounding matrix that proves the correct classification of the logs.

```
Random Forest Accuracy:  0.9929034555285974
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    116563
           1       0.92      0.87      0.90      4200

    accuracy                           0.99    120763
   macro avg       0.96      0.94      0.95    120763
weighted avg       0.99      0.99      0.99    120763


confusion_matrix(Y_train, y_pred)
```

Figure 8 - Accuracy

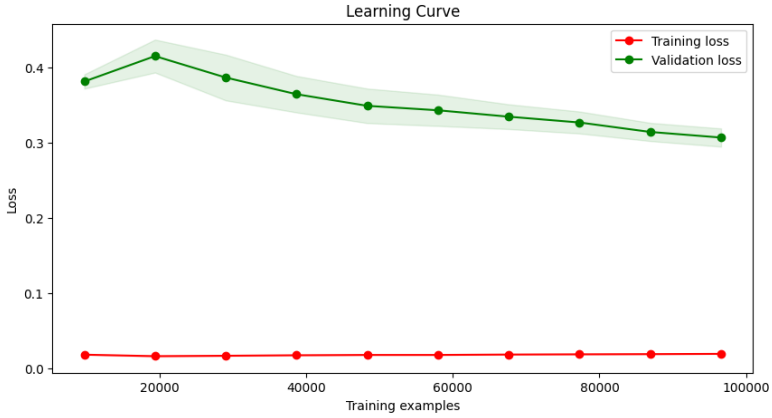

Figure 9 - Learning Curve

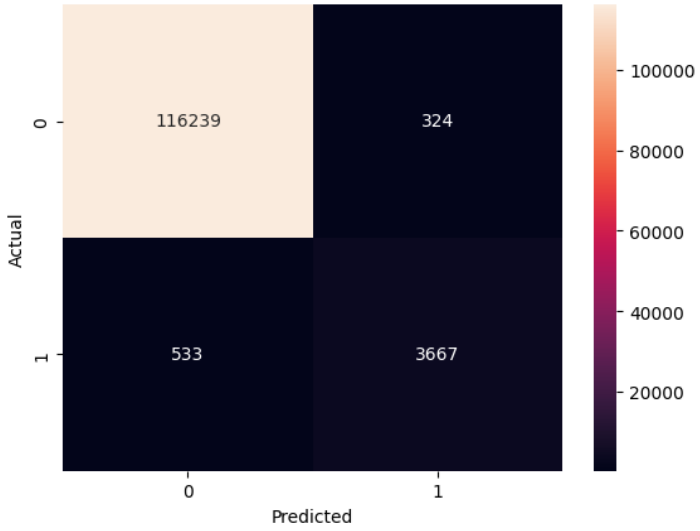

Figure 10 - Confusion Matrix

Based on these images, it can be seen that the accuracy is very good, 99%. The accuracy, recall, and F1-Score are very close to 1, the maximum value, but not wanting it to reach 1, as it could be signs of overfitting. The training curve graph also shows good performance, as the training loss is low and the training validation goes down throughout the training process.

4.3.2 XGBoost Training

XGBoost (Extreme Gradient Boosting) is a highly efficient ML algorithm based on the gradient boosting technique. This technique builds strong predictive models by combining several weak models, usually decision trees, and adding them sequentially. Each new tree is trained to correct the errors made by the previous ones, optimizing a loss function through gradients, which results in a more accurate and robust model. XGBoost is known for its high performance, computational efficiency, and its ability to handle incomplete and unbalanced data.

After this initial experience with Random Forest, XGBoost was tested, which, although showing good results, did not outperform Random Forest in predicting certain patterns.

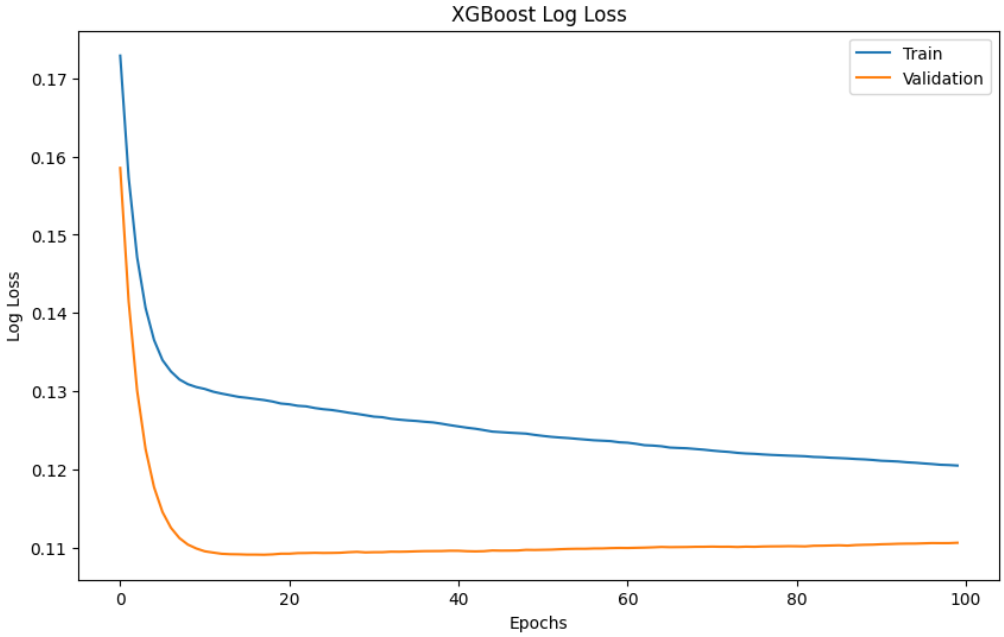

Figure 11 - XGBoost Learning Curve

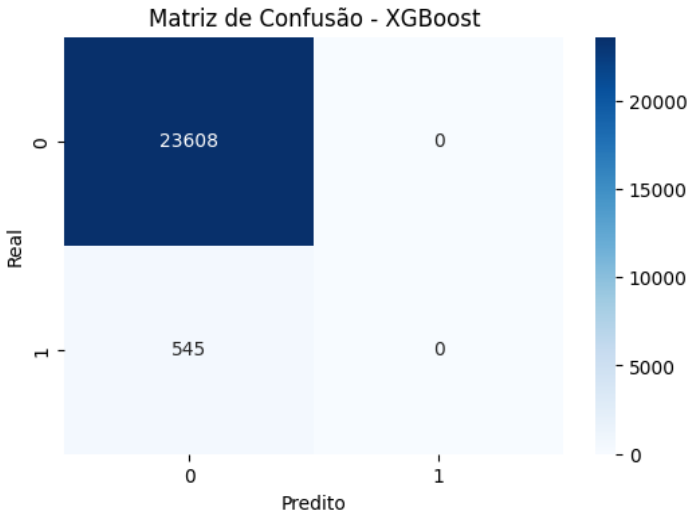

Figure 12 - XGBoost Confusion Matrix

```
XGBoost Accuracy: 0.9774
              precision    recall  f1-score   support

           0       0.98      1.00      0.99     23608
           1       0.00      0.00      0.00       545

    accuracy                           0.98     24153
   macro avg       0.49      0.50      0.49     24153
weighted avg       0.96      0.98      0.97     24153
```

Figure 13 - XGBoost Accuracy

According to the images, it can be seen that the accuracy is quite good, almost 98%, which at first glance looks promising. However, the accuracy, recall and F1-Score, for class '1', are all 0. You are having difficulty correctly predicting the '1' classes, which in this case is where the labels that have been defined exist. It may be because there is an imbalance in the data, where class '1' is underrepresented in relation to class '0', which leads the model to favor the majority.

Additionally, the loss graph suggests that training loss and validation go down over time, which is good. But validation stabilizes after 20 epochs and training continues to drop slightly. This pattern suggests that the model may be overfitting the data, starting to overfit and losing the ability to generalize.

These observations demonstrate that while XGBoost is a powerful algorithm, its effectiveness depends heavily on the quality and balance of the input data. As such, the importance of careful data preparation and the choice of evaluation metrics is reinforced, especially when working with unbalanced classes.

4.3.3 KMeans

KMeans is an unsupervised learning algorithm widely used for clustering tasks, i.e., to group data into subsets with similar characteristics. Its main objective is to divide a dataset into k distinct clusters, based on the similarities between the data. It is

especially useful when we want to explore hidden patterns or structures in the data without prior knowledge of the categories.

The algorithm works in the following steps:

- Initialization: k points are randomly selected from the data space, which will serve as initial centroids.

- Attribution: Each data point is assigned to the cluster whose centroid is closest, based on a distance metric (Euclidean distance).

- Update: new centroids are calculated, updating their position to the average of the points assigned to each cluster.

- Iteration: The assignment and update steps are repeated until the centroids stop changing significantly or reach a maximum number of iterations.

This process usually converges towards a stable solution, although it does not guarantee to find the overall optimal grouping.

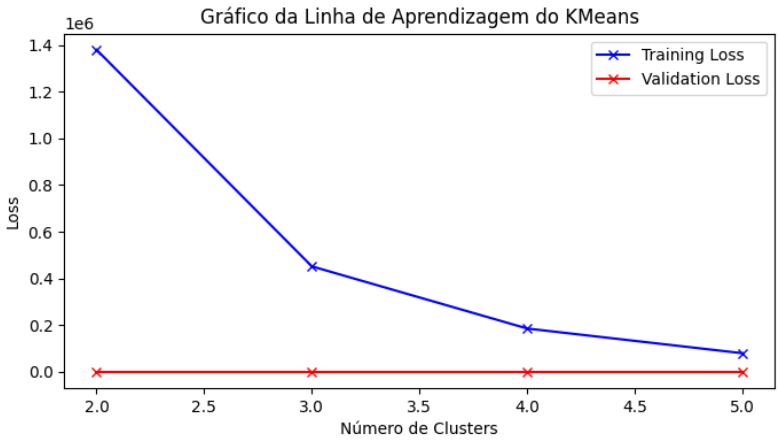

Figure 14 - KMeans Learning Graph

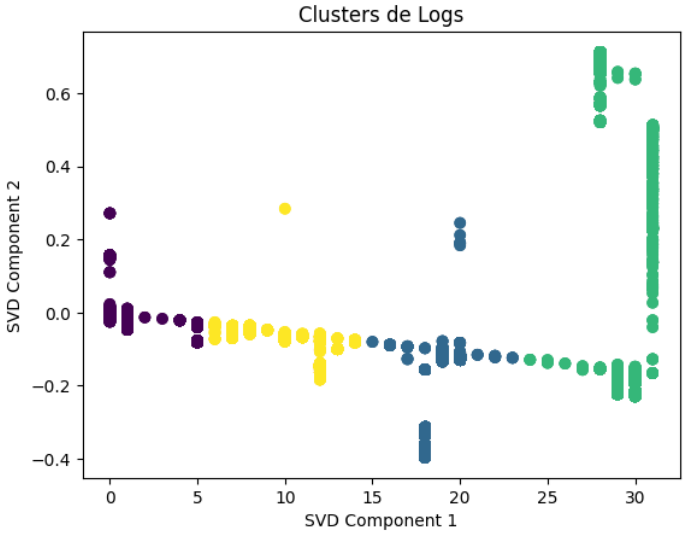

Figure 15 - KMeans Clusters

In the first image, you can see the KMeans Learning Line Graph applied to the data, where the Training Loss decreases as the number of clusters increases. This behavior is expected, since with more clusters, the model adjusts better to the data. However, there is no significant loss in Validation Loss, which remains virtually constant and close to zero. This indicates that the model may be overfitting the training data, or not reading well with data that is very different from each other.

The choice of the number of clusters is one of the main limitations of KMeans, as it must be defined in advance, which can lead to arbitrary or incorrect segmentation of data. There is no automatic way to determine the optimal number of clusters, which makes the process dependent on heuristics or successive tests, as seen in this graph.

The second image shows the distribution of the clustered data, using a dimensionality reduction by SVD (Singular Value Decomposition). Visually it is possible to identify the formation of groups, but their separation may not accurately reflect the actual structure of the data, especially when it contains noise or is distributed in a non-spherical way, something to which KMeans is particularly sensitive.

Due to these limitations, the DBSCAN algorithm was also explored, which proved to be more robust, as it does not require the previous definition of the number of clusters and for better dealing with noise and clusters in arbitrary ways. DBSCAN automatically identifies groups based on data density, making it a more suitable option for data such as logs from systems, where the internal structure can be complex and irregular.

4.3.4 DBSCAN

DBSCAN is a density-based clustering algorithm that groups data points that are compressed and marks outliers as noise based on their density in feature space. Identifies clusters as dense regions in the data space.

Unlike KMeans or hierarchical clustering, which assume that clusters are compact and spherical, DBSCAN excels at handling real-world data irregularities. Clusters can take any shape and not just circular or convex. It also effectively identifies and manipulates noise points without assigning them to any cluster.

DBSCAN identifies the clusters and does not need to specify the number of clusters in advance.

Vary the eps varies the radius of the neighborhood. If the distance between two points is less than or equal to eps, they are considered neighbors. If the eps is too small, most of the points will be classified as noise. If it is too large, the clusters may merge and the algorithm may not be able to identify them. You can use the distance graph K to determine the eps.

DBSCAN works by categorizing data points into three types:

- Central, which have a sufficient number of neighbors within a specified radius (eplison).

- Boundary points, which are close to hub points but don't have enough neighbors to be hub points themselves.

- Noise points, which do not belong to any cluster.

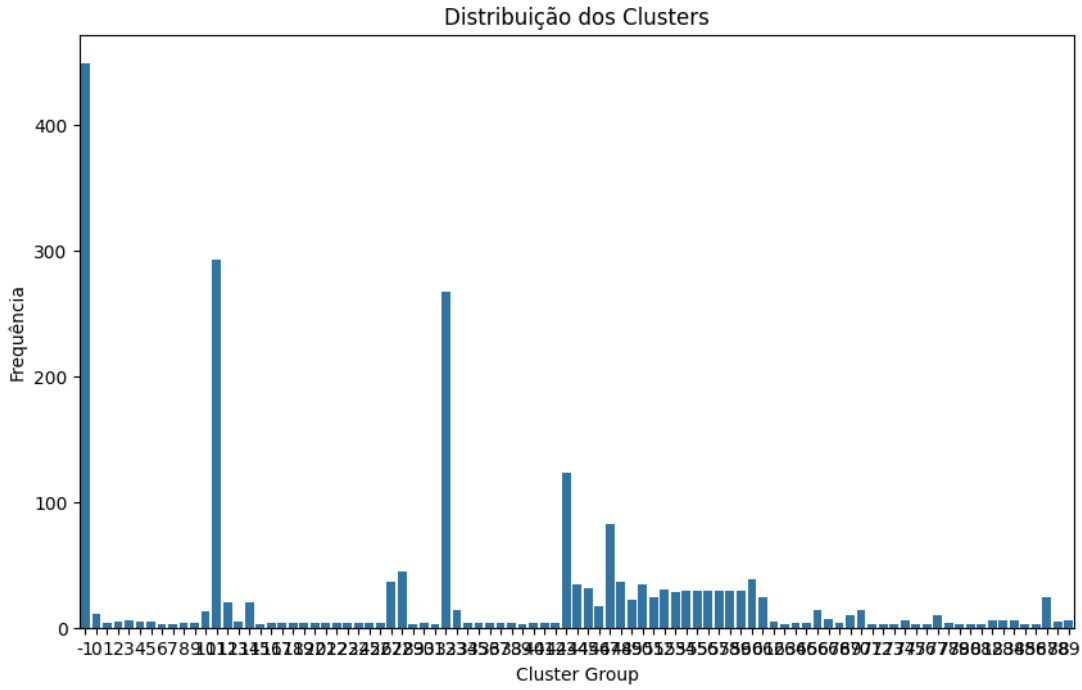

Figure 16 - DBSCAN Clusters Distribution

The graph above shows the distribution of the groups identified by the algorithm. Each bar represents a distinct cluster, while the frequency indicates the number of samples assigned to that group.

It is possible to observe:

- Group -1, on the left, represents the noise points, that is, instances that do not belong to any cluster because they do not meet the density criteria. DBSCAN handles these cases natively, without forcing them to integrate into any group.

- There is a wide variety of clusters identified, with different sizes, reflecting DBSCAN's ability to detect patterns of different shapes and densities, something that KMeans cannot do as much, since it assumes spherical shapes and requires the number of groups to be defined in advance.

- This granularity allows for richer analysis of the data, especially useful in contexts such as test logs, where event patterns and errors do not follow regular or predictable forms.

Together with the tests done and the visual observation of clusters (as in the scatter plot), this histogram validates the effectiveness of DBSCAN in segmenting complex and heterogeneous data.

After these tests and understanding which would be the best algorithm, the conclusion was reached as to which algorithms would have to be worked on. In this case it would be Random Forest and DBSCAN, a semi-supervised model.

4.3.5 API Development

The next step to be taken was the development of an API and a graphical interface. We started by designing the structure of the architecture to be implemented, as it was known that it would be appropriate to use a Model-View-Controller (MVC) and Microservices architecture. The decision of this aimed to ensure the division of responsibilities between the components of the system, facilitate maintenance, promote scalability and evolution of the project over time.

The chosen architecture is composed of three major main components: backend (Application Programming Interface (API)), frontend (user interface) and natural language service (LLM). Each of the components was developed as an independent microservice, communicating with the others through well-defined interfaces. In order to ensure portability and isolation between services, all components were containerized using Podman.

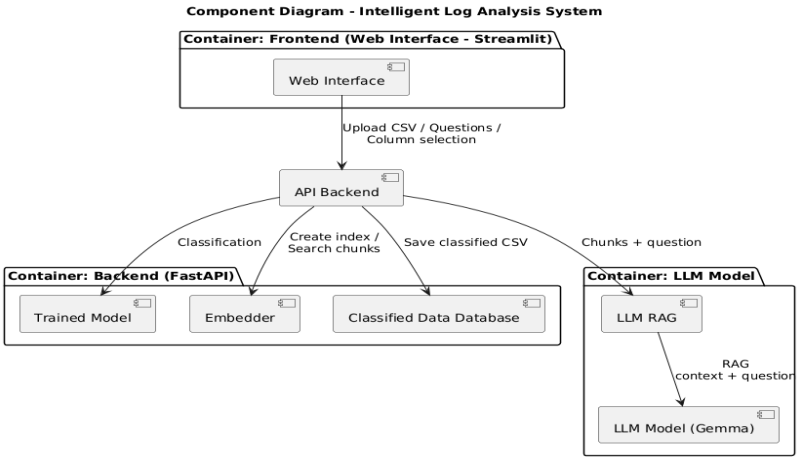

Figure 17 - Project Architecture

4.3.5.1 Backend- API e Processing

The backend was developed using the FastAPI framework, which offers a modern, efficient and high-performance structure for building RESTful APIs (interface that two computer systems use to exchange information securely over the internet). This service has several responsibilities within the architecture such as:

- Allow the user to upload CSV files.
- Apply the trained ML models to the processed data.
- Generate and save the output files, which can later be used for analysis.
- Communicate with the database for storage and retrieval of vector embeddings.
- Make endpoints available for integration with the LLM service and frontend.

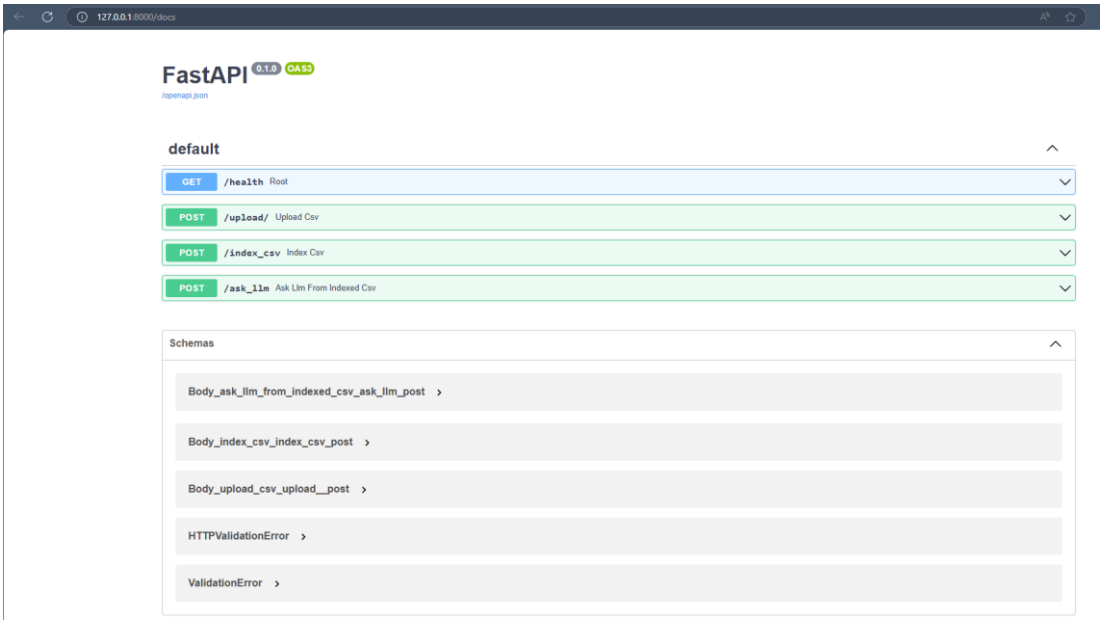

Figure 18 - FastAPI Framework

Method	Endpoint	Description
GET	/health	Check the status of the API
POST	/upload/	Upload a CSV file
POST	/index_csv	Index the uploaded CSV
POST	/ask_llm	Send a question to the LLM based on the indexed CSV

4.3.6 LLM Service – RAG

A micro-service dedicated to integration with a language model was also developed, in this case Ollama with a locally executed gemma model, due to the permanence of data privacy. This service allows users to interact with processed files through natural language questions.

To make the research more efficient and relevant, a RAG approach was implemented. It combines the generative power of the language model with embedding-based information retrieval, using HuggingFace as a vector indexing mechanism. CSV files are chunked, indexed, and user queries are compared to existing embeddings to provide contextualized answers based on the actual data.

4.3.7 Frontend – User Interface

The frontend was developed using Streamlit, which allows a simple and effective interface where the user can:

Upload CSV files for analysis

Index files and choose the columns you want to generate embeddings

- Interact with LLM

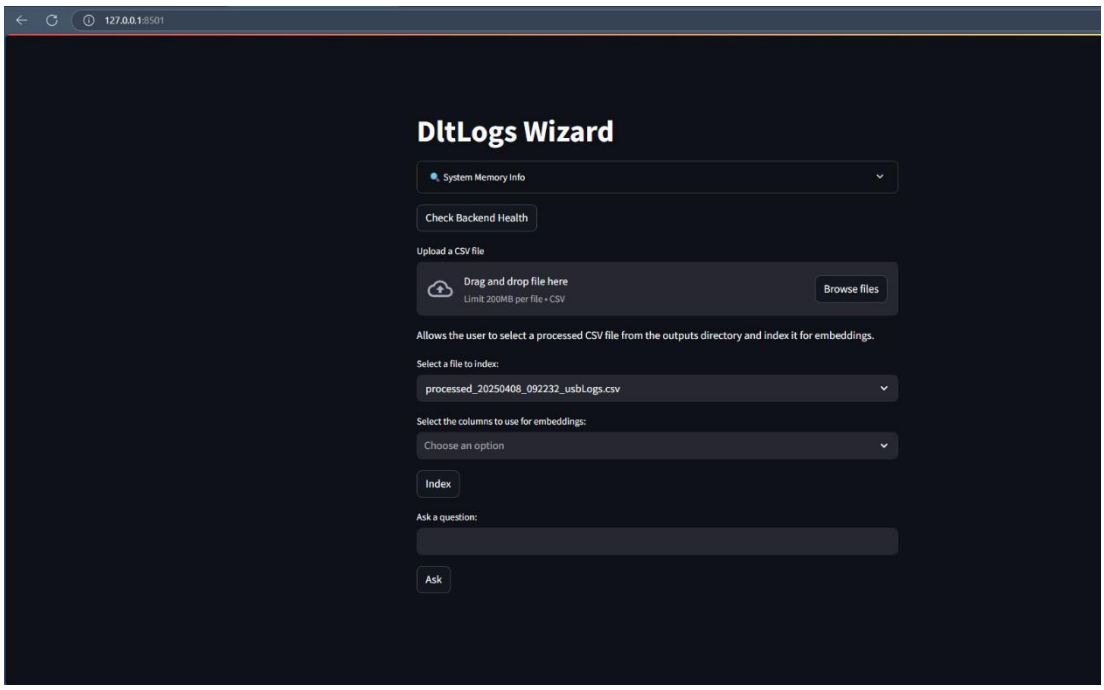

Figure 19 - User Interface

This separation between the backend and frontend ensures that any change in the processing logic does not affect the presentation layer, thus following the principles of modular development.

4.3.8 Containerization with Podman

To facilitate the execution of the project in different environments and avoid dependency conflicts, containerization with Docker was used. Each component (backend, LLM, and frontend) runs in an isolated container, with its configuration, dependencies, and runtime environment. In addition to simplifying the deployment process, this approach also allows each service to be scaled independently, if necessary. For example, the LLM can run in an environment with a GPU, while the other services remain on conventional servers.

4.3.9 PGVector Database – Embeddings Storages

To allow for efficient management of the vector embeddings generated from the CSV files, a PostgreSQL database with the pgvector extension was integrated into the backend container. This database stores representative vectors of the contents of the files, enabling efficient vector searches on the data later. The pgvector extension is essential for maintaining the performance of the RAG system, as it allows direct comparisons between vectors within the database, without the need for full loading into memory. The integration between the database and the backend also provides for maintaining a history of the analyzed files, along with their respective vectors, facilitating future analyses, data reuse, and continuous improvement of the system.

4.3.10 Long Short-Term Memory (LSTM)

In addition to the traditional Machine Learning models applied in the initial phase of the project, a complementary study was developed with Deep Learning models, more specifically a Recurrent Neural Network (RNN) of the LSTM type. This model was used to classify log blocks as fault-related (CRSH) or not, based on the textual context present in the column where the information is located.

Prior to the implementation of LSTM, it was necessary to properly prepare the data. The nominal variables were treated using the Categorical method of the pandas library, which allowed the conversion of the values into explicit categories,

maintaining a clean and structured representation. The resulting file was saved and served as the basis for the following models.

In the Deep Learning part, consecutive log blocks were aggregated, based on the alternation of CRSH lines, to construct textual sequences representative of each block. Next, a text cleanup and normalization function was applied, removing redundant symbols and spaces.

Next, TensorFlow's Tokenization and Padding API was used to prepare the textual data for the model. The text was converted into sequences of integers and standardized to the same length (max_len = 200). To deal with class imbalance, weights were calculated by classes that were applied during training.

The LSTM model was composed of an embedding layer, followed by an LSTM layer with 32 units, and Dropout layers for regularization. After five seasons of training, a good performance was achieved in terms of accuracy and loss, both in training and validation. Finally, the model and tokenizer were saved for later use and integration.

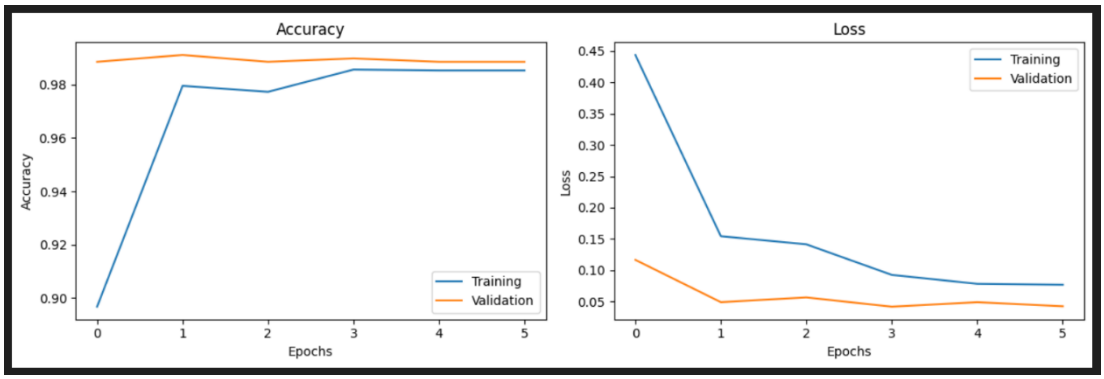

Figure 20 - LSTM Learning Curve

This study revealed that the use of Deep Learning models can complement the RAG-based system, offering a more robust approach to automatic log classification based only on textual content. Such an approach is especially useful when you don't want to rely on semantic indexing or when you want a more autonomous and specialized solution for fault detection.

A significant advantage of using the LSTM model instead of Random Forest lies in its ability to understand text strings with temporal or contextual dependencies. While Random Forest operates on isolated tabular features, LSTM specializes in sequential data, such as the textual content of the column that contains the main information. This allows LSTM to detect more complex patterns in the message flow of the logs, considering the order and relationship between tokens over time, something that tree-based models cannot capture. This property makes LSTM particularly effective in log analysis, where sequential context can be determined to identify failures.

4.3.11 Obtained Results

After applying the different algorithms and approaches, it was possible to obtain relevant results regarding the detection of patterns and errors in logs in the automotive industry.

Initially, several supervised and unsupervised ML algorithms were tested. It was concluded that the most suitable for this type of data would be Random Forest (supervised classifier) and DBSCAN (unsupervised grouping). Both were trained and evaluated based on a real data set, with all error situations and critical situations in the logs.

The Random Forest model proved to be effective in classifying error events, having been trained on previously processed data, with vectorization of the relevant columns and normalization of the numerical variables. Its ability to deal with noise and class imbalance was useful for the project's objectives, as can be seen in the following image. The model was saved in a file to then be used in the development phase with new data and be implemented in the development architecture.

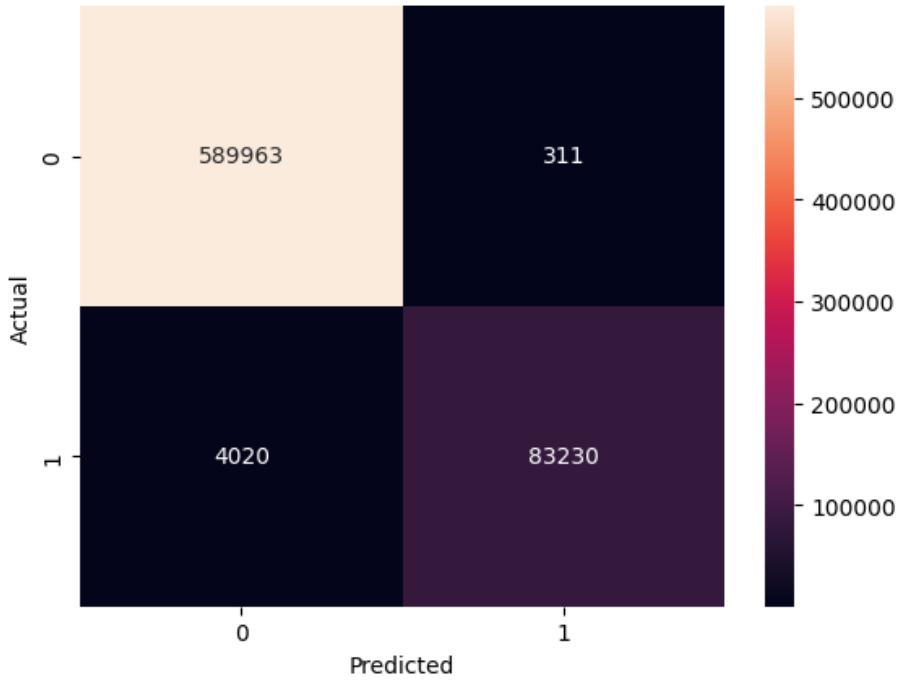

Figure 21 - Random Forest Result

DBSCAN was used to identify groupings of events based on characteristics extracted from the logs. This algorithm demonstrated a clear advantage over other methods such as K-Means, as it does not assume the existence of spherical clusters and is able to detect patterns arbitrarily, respecting the irregular nature of the real data. An example of this effectiveness can be seen in the cluster distribution graph, where DBSCAN's ability to identify dense groups and distinguish noise points stands out.

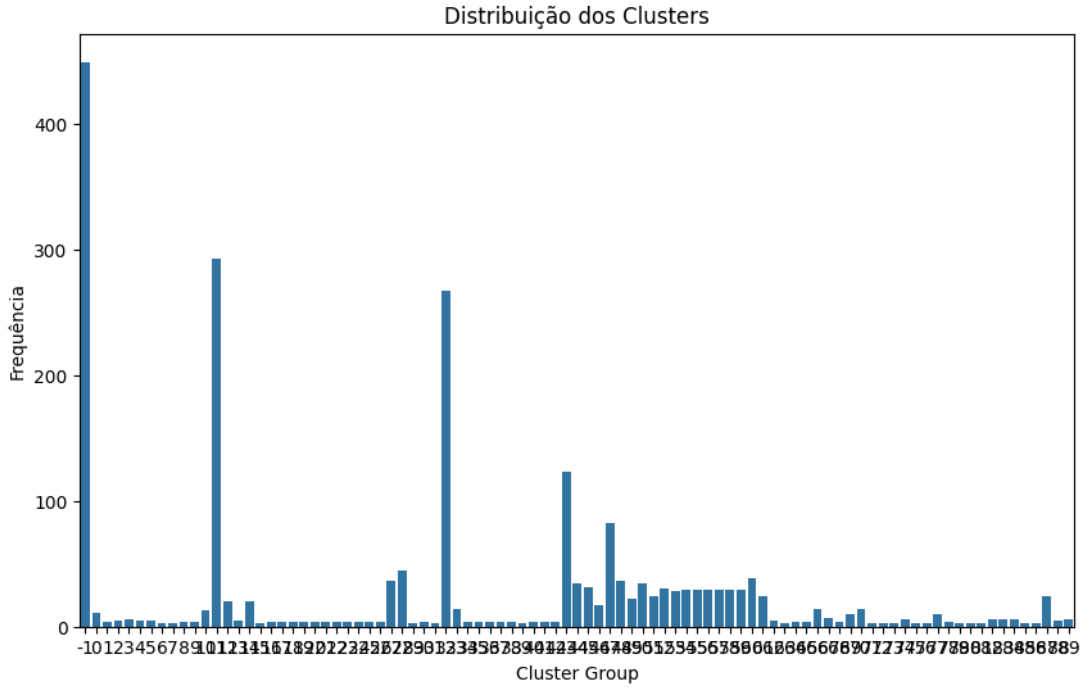

Figure 22 - DBSCAN Clusters Result

This type of distribution confirms that DBSCAN was able to detect multiple relevant clusters, even with noisy and high-dimensional data.

A RAG system has also been implemented with the aim of allowing the user to ask questions in natural language about the processed data. The system works as follows:

- The data from the CSVs is divided into chunks and converted into vector embeddings.
- These embeddings are indexed with the HuggingFaceEmbeddings from the LangChain library.
- When it receives a question, the system retrieves the most relevant chunks and provides that context to the LLM.
- LLM responds based on the content of the data, facilitating AI-assisted interpretation.

This feature was very useful in analyzing large volumes of logs, allowing the team to get quick answers about patterns, versions, errors, and other parameters without the need for additional programming.

To support the entire workflow, a modular structure based on the MVC standard was developed, distributed in three main components, each running in a Podman container:

- Backend (FastAPI), responsible for receiving CSV files, processing the data and applying the trained models.
- Frontend (Streamlit), allows the user to interact with the application, upload and consult the outputs in a simple and intuitive way.
- LLM Local (via Ollama), a container dedicated to running an offline LLM, such as gemma, that answers questions about the data.

Thus, with this structure and separation, it is possible to maintain and scale the system in a simple way.

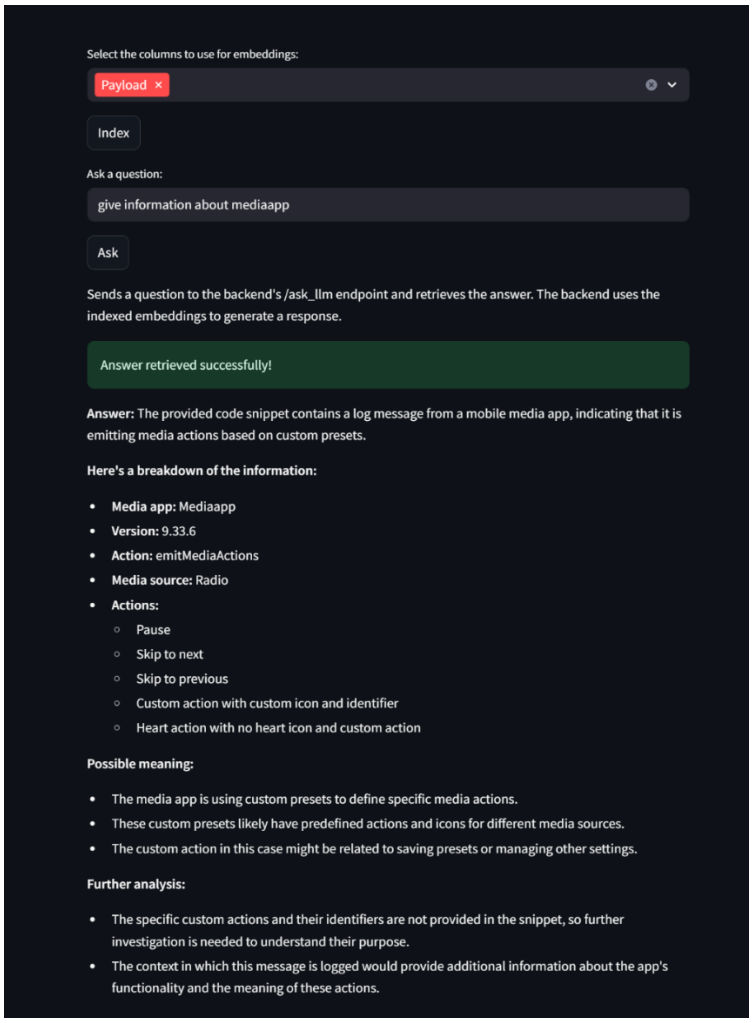

Figure 23 - Semantic Search

Later, and on the recommendation of an expert in artificial intelligence, the use of Deep Learning models was also explored, namely recurrent neural networks such as LSTM applied to textual columns. The goal was to detect patterns and anomalies in text sequences, complementing the classical vectorization-based approach. These models demonstrate potential in the semantic and sequential analysis of the system's feedback, as seen in the following image, which shows the training carried out, and may be integrated in the future into a more robust NLP pipeline.

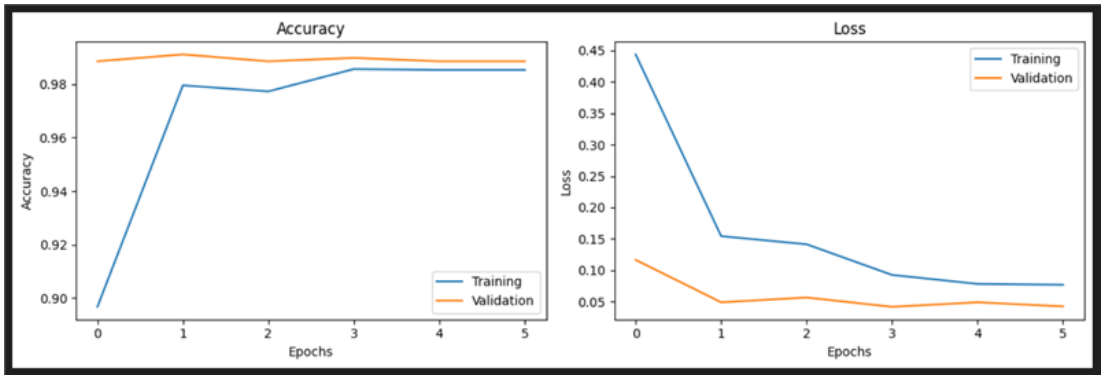

Figure 24 - LSTM Accurary

5 Conclusion

This project, developed during an internship at Critical Software, aimed to design and implement an intelligent tool for log analysis in the automotive industry. The main motivation originated from the need to process large volumes of logs generated by automated testing systems. Log analysis represents a labor-intensive, error-prone activity that is difficult to scale manually.

To address this challenge, the proposed solution successfully combined ML techniques, including supervised classification and unsupervised clustering, with a RAG approach. By integrating a locally hosted LLM and providing a natural language interface, the system enables semantic search over pre-processed logs, facilitating the rapid detection of errors, patterns, and meaningful insights.

Throughout the project, the initially defined objectives were achieved effectively. The resulting tool is functional, modular, and extensible, capable of accurately identifying anomalies and providing coherent, context-aware responses using LLM. The development process included several key stages:

- Data preprocessing and feature engineering.
- Model training using classification and clustering.
- Implementation of a RAG pipeline using HuggingFace, and Ollama.
- Development of a web-based interface with FastAPI and Streamlit.
- Containerization and deployment using Podman.

This work also consolidated a wide set of skills acquired during the degree, particularly in machine learning, backend and frontend development, API design, and containers orchestration. The use of modern frameworks such as TensorFlow, HuggingFaceTransformers, LangChain, and Ollama contributed to a deepening of

technical Knowledge. In addition, transversal competencies such as problem-solving, adaptability, critical thinking, and time management were reinforced.

Despite the positive outcomes, several challenges were encountered:

- Limited computational resources posed difficulties in training more complex models and running concurrent services.
- The development cycle often preceded the formal definition of requirements, which occasionally hampered planning and technical documentation.
- The system, although robust, could benefit from further optimization in performance, more granular access controls, and extended evaluation metrics for the models used.

At the end of the internship, the final version of the tool demonstrated solid performance in real use cases, but also revealed areas for future improvement that will be explained further ahead.

In summary, this internship provided a highly valuable experience, merging theoretical knowledge with practical skills in an industry-relevant context. It strengthened the foundation for further professional development in the areas of AI, data analysis, and intelligent systems engineering, while also encouraging a more mature, product-oriented approach to software development.

5.1 Future Work

The results obtained in this project were positive and clearly demonstrate the feasibility of the proposed approach. However, there are areas that could be explored differently and improved in future work, both functionally and technically.

With access to more robust computational resources, such as distributed environments, Cloud infrastructures, or GPU-equipped servers. Future implementations could enable:

- More complex training processes
- Efficient execution of LLMs (Large Language Models)

- Optimization of tasks like text vectorization
- Use of larger datasets
- More scalable and fluid container management

This infrastructure advancement would allow integrating larger LLMs with more parameters and enhanced contextual comprehension capabilities, elevating the system's overall performance. Such improvements would grant significantly faster, more precise, and comprehensive log analysis, enhancing automation quality and extraction of insights from large data volumes.

5.2 Critical Reflections

One of the main challenges faced throughout the development of this project was related to the computational requirements and associated with the training of machine learning models, especially in clustering tasks applied to log analysis. The processing of large volumes of data combined with text vectorization and the training of algorithms such as DBSCAN, proved to be demanding in terms of memory and processing capacity.

During the internship, these limitations forced several commitments to be made, such as reducing the size of the datasets to be representative samples and simplifying certain processes to enable training in the available resources. This condition directly affected the execution time, the depth of the analyses, and the ability to experiment with more complex architectures or with a greater number of parameters.

In addition to the challenges related to training ML models, the execution of LLM models, as well as the process of generating embeddings of the files and saving them in a database, proved to be equally resource demanding. Running an LLM locally through Ollama, even using optimized models such as Gemma, often caused the computer's Random Access Memory (RAM) to run out. Managing multiple containerized microservices simultaneously, including the API, frontend, and embedding service, further aggravated the computational load, thus making development and testing more complex and time-consuming.

Another point that could have been better conducted was the approach adopted in the development of the project. The work began with the implementation of the system and only then construction of the requirements and respective diagrams. This sequence, although functional, does not follow the best practices recommended in software engineering, where the structured and concise definition of requirements must precede any implementation phase.

Ideally, the correct way of implementing the project should have followed a more planning-oriented approach, starting with requirements analysis, defining use cases and elaborating supporting diagrams (such as class, sequence or component diagrams). Only after this phase would it be appropriate to move on to implementation.

This inversion in the process made it difficult, at times, to make technical decisions, structure the solution and clearly communicate the objectives to be achieved.

References

[1] *Critical Software | Mission and Business Critical Bespoke Software Development*. (n.d.). Https://Www.Criticalsoftware.Com/En.

[2] Gholamian, S., & Ward, P. A. S. (2021). A Comprehensive Survey of Logging in Software: From Logging Statements Automation to Log Mining and Analysis. *Arxiv.org*. https://doi.org/10.48550/arXiv.2110.12489

[3] Mariño, J. (2024, July 29). *Fraunhofer IESE*. Fraunhofer IESE. https://www.iese.fraunhofer.de/blog/feral-testbed-implementation-of-an-infotainment-system-for-validation-and-testing/

[4] *Copeland, B. (2025, May 22). artificial intelligence. Encyclopedia Britannica. https://www.britannica.com/technology/artificial-intelligence*

[5] Crabtree, M., & Nehme, A. (2023, July 10). *What is Data Science? Definition, Examples, Tools & More*. Datacamp.com; DataCamp. https://www.datacamp.com/blog/what-is-data-science-the-definitive-guide?

[6] Miser, E., & Sarioguz, O. (2024). DATA-DRIVEN DECISION-MAKING: TRANSFORMING MANAGEMENT IN THE INFORMATION AGE. *International Research Journal of Modernization in Engineering Technology and Science*, *6*(2). https://doi.org/10.56726/irjmets49577

[7] Subrahmanya, S. V. G., Shetty, D. K., Patil, V., Hameed, B. M. Z., Paul, R., Smriti, K., Naik, N., & Somani, B. K. (2021). The role of data science in healthcare advancements: applications, benefits, and future prospects. *Irish Journal of Medical Science*, *191*(4). https://doi.org/10.1007/s11845-021-02730-z

[8] *What Is Machine Learning (ML)?* (n.d.). Https://Www.Paloaltonetworks.Co.Uk/Cyberpedia/Machine-Learning-Ml.

[9] What Is Supervised Learning? | IBM. (2024). In *https://www.ibm.com/think/topics/supervised-learning*.

[10] Keita, Z. (2022, September 21). *Classification in Machine Learning: An Introduction*. Datacamp.com; DataCamp. https://www.datacamp.com/blog/classification-machine-learning?

[11] Afriyie, J. K., Tawiah, K., Pels, W. A., Addai-Henne, S., Dwamena, H. A., Owiredu, E. O., Ayeh, S. A., & Eshun, J. (2023). A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions. *Decision Analytics Journal*, *6*(100163), 100163. https://doi.org/10.1016/j.dajour.2023.100163

[12] Tripathi, A. (2025, March 22). *Supervised Learning for Image Recognition: How It Works*. DEV Community. https://dev.to/aditya_tripathi_17ffee7f5/supervised-learning-for-image-recognition-how-it-works-5h33

[13] K. Sashi Rekha, T. Amutha, R. Usharani, S. Pushparani, & V.M. Sivagami. (2024). *Predictive Analysis of Consumer Behavior Using Supervised Learning Techniques*. 1–6. https://doi.org/10.1109/ic3tes62412.2024.10877475

[14] *Supervised Learning: Definition, Techniques & Benefits*. (2025, January 8). Lyzr. https://www.lyzr.ai/glossaries/supervised-learning/?

[15] *What Is Unsupervised Learning? | IBM. (2021, September). Https://Www.Ibm.Com/Think/Topics/Unsupervised-Learning*.

[16] Karl, T. (2024, February 12). *DBSCAN vs. K-Means: A Guide in Python*. New Horizons. https://www.newhorizons.com/resources/blog/dbscan-vs-kmeans-a-guide-in-python?

[17] *What is Kubeflow Pipelines?* (2023, September 12). Iguazio. https://www.iguazio.com/glossary/unsupervised-ml/?

[18] GeeksforGeeks. (2017, June). *Introduction to Dimensionality Reduction*. GeeksforGeeks. https://www.geeksforgeeks.org/dimensionality-reduction/?

[19] Sagar Shankaran. (2024, October 4). *Unsupervised learning is a branch of machine learning that deals with unlabeled data, aiming to discover hidden patterns or intrinsic structures without predefined outputs. Unlike supervised learning, it doesn't rely on labeled datasets but instead seeks to understand the data's underlying structure*. Linkedin.com. https://www.linkedin.com/pulse/unsupervised-learning-its-application-sagar-shankaran-mutuc/

[20] *What is Kubeflow Pipelines?* (2023, September 12). Iguazio. https://www.iguazio.com/glossary/unsupervised-ml/?

[21] *What Is Semi-Supervised Learning? | IBM*. (2023). Https://Www.Ibm.Com/Think/Topics/Semi-Supervised-Learning.

[22] Team, A. E. (2024, March 29). *Semi-Supervised Learning, Explained with Examples*. AltexSoft. https://www.altexsoft.com/blog/semi-supervised-learning/?

[23] team, I. editorial. (2025, April 2). *What is semi-supervised learning?* IONOS Digital Guide; IONOS. https://www.ionos.ca/digitalguide/online-marketing/search-engine-marketing/semi-supervised-learning/?

[24] Team, A. E. (2024, March 29). *Semi-Supervised Learning, Explained with Examples*. AltexSoft. https://www.altexsoft.com/blog/semi-supervised-learning/?

[25] *Reinforcement Learning - GeeksforGeeks*. (2025). Https://Www.Geeksforgeeks.Org/What-Is-Reinforcement-Learning/.

[26] Adithya Prasad Pandelu. (2024, December 28). *Day 62: Reinforcement Learning Basics — Agent, Environment, Rewards*. Medium. https://medium.com/%40bhatadithya54764118/day-62-reinforcement-learning-basics-agent-environment-rewards-306b8e7e555c

[27] Pranjal Khadka. (2024, March 31). *A Brief Overview to Reinforcement Learning - Pranjal Khadka - Medium*. Medium. https://medium.com/%40pranjalkhadka/a-brief-overview-to-reinforcement-learning-10ec6b517eb7

[28] GeeksforGeeks. (2018, April 25). *Reinforcement Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/what-is-reinforcement-learning/?

[29] Deepchecks Community Blog. (2023, April 18). *Reinforcement Learning Applications: From Gaming to Real-World*. Deepchecks. https://www.deepchecks.com/reinforcement-learning-applications-from-gaming-to-real-world/

[30] Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, *12*, 100470. https://doi.org/10.1016/j.mlwa.2023.100470

[31] *Natural Language Processing (NLP) - A Complete Guide*. (2023, January 11). Deeplearning.ai. https://www.deeplearning.ai/resources/natural-language-processing/?

[32] (PDF) Big Data Analytics in Supply Chain: A Literature Review. (n.d.). In *https://www.researchgate.net/publication/327979282_Big_Data_Analytics_in_Supply_Chain_A_Literature_Review*.

[33] Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, *12*, 100470. https://doi.org/10.1016/j.mlwa.2023.100470

[34] Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, *12*, 100470. https://doi.org/10.1016/j.mlwa.2023.100470

[35] Sites, P. (2025, March 8). *How to Analyze Logs Using AI*. LogicMonitor. https://www.logicmonitor.com/blog/how-to-analyze-logs-using-artificial-intelligence?

[36] O'Donnell, J. (2024, December 13). *AI Log Analysis – Shaping the Future of Observability*. Logz.io. https://logz.io/blog/ai-log-analysis/?

[37] Marche, S. (2024, August 23). *Was Linguistic A.I. Created by Accident?* The New Yorker. https://www.newyorker.com/science/annals-of-artificial-intelligence/was-linguistic-ai-created-by-accident?

[38] *What is a Large Language Model (LLM) - GeeksforGeeks*. (n.d.). Https://Www.Geeksforgeeks.Org/Large-Language-Model-Llm/.

[39] Stöffelbauer, A. (2025, February 21). How Large Language Models Work. From zero to ChatGPT | by Andreas Stöffelbauer | Medium | Data Science at Microsoft. *Medium*. https://medium.com/data-science-at-microsoft/how-large-language-models-work-91c362f5b78f

[40] Keary, T. (2023, July 14). *12 Practical Large Language Model (LLM) Applications*. Techopedia. https://www.techopedia.com/12-practical-large-language-model-llm-applications

[41] Calma, J. (2025, May 29). *AI could consume more power than Bitcoin by the end of 2025*. The Verge. https://www.theverge.com/climate-change/676528/ai-data-center-energy-forecast-bitcoin-mining?

[42] Bai, X., Wang, A., Ilia Sucholutsky, & Griffiths, T. L. (2025). Explicitly unbiased large language models still form biased associations. *Proceedings of the National Academy of Sciences*, *122*(8). https://doi.org/10.1073/pnas.2416228122

[43] *LLM-SAP: Large Language Models Situational Awareness-Based Planning*. (2022). Arxiv.org. https://arxiv.org/html/2312.16127v5?

[44] *What is RAG? - Retrieval-Augmented Generation AI Explained - AWS*. (n.d.). Amazon Web Services, Inc. https://aws.amazon.com/what-is/retrieval-augmented-generation/

[45] GeeksforGeeks. (2025a, January 27). *Clustering in machine learning*. GeeksforGeeks. https://www.geeksforgeeks.org/clustering-in-machine-learning/

[46] GeeksforGeeks. (2025a, January 20). *Getting started with Classification*. GeeksforGeeks. https://www.geeksforgeeks.org/getting-started-with-classification/

[47] Meta. (n.d.). *FAISS*. Ai.meta.com. https://ai.meta.com/tools/faiss/

[48] 🤗 *Transformers*. (n.d.-b). https://huggingface.co/docs/transformers/v4.17.0/en/index

[49] *pandas - Python Data Analysis Library*. (n.d.). https://pandas.pydata.org/about/index.html

[50] GeeksforGeeks. (2025c, March 26). *Python NumPy*. GeeksforGeeks. https://www.geeksforgeeks.org/python-numpy/

[51] GeeksforGeeks. (2025c, March 17). *MatPlotLib Tutorial*. GeeksforGeeks. https://www.geeksforgeeks.org/matplotlib-tutorial/

[52] GeeksforGeeks. (2023, February 28). *What is PostgreSQL Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/what-is-postgresql-introduction/

[53] *What is Python? Its Uses and Applications - GeeksforGeeks*. (n.d.). Https://Www.Geeksforgeeks.Org/What-Is-Python/.

[54] *What is Visual Studio Code? Microsoft's extensible code editor*. (n.d.). Https://Www.Infoworld.Com/Article/2335960/What-Is-Visual-Studio-Code-Microsofts-Extensible-Code-Editor.Html.

[55] *What is the Jupyter Notebook?¶*. (n.d.). Https://Jupyter-Notebook-Beginner-Guide.Readthedocs.Io/En/Latest/What_is_jupyter.Html.

[56] Gosselin, E. (n.d.). *PlantUML for database modeling*. Https://Medium.Com/@elvis.Gosselin/Plantuml-for-Database-Modeling-1b71e6d4622d.

[57 Atlassian. (n.d.). *Bitbucket Overview | Bitbucket*. Https://Bitbucket.Org/Product/Guides/Getting-Started/Overview#a-Brief-Overview-of-Bitbucket.

[58] *What is Podman Compose and How Does it Work?* (n.d.). Https://Supportfly.Io/Podman-Compose/.

[59] *FastAPI*. (n.d.). Https://Fastapi.Tiangolo.Com/.

[60] *Streamlit. (n.d). A faster way to build and share data apps*. Retrieved from https://streamlit.io/

[61] GeeksforGeeks. (2025, May 27). *Learning model building in Scikitlearn*. GeeksforGeeks. https://www.geeksforgeeks.org/learning-model-building-scikit-learn-python-machine-learning-library/

[62] *What is LangChain? - LangChain Explained - AWS*. (n.d.). Amazon Web Services, Inc. https://aws.amazon.com/what-is/langchain/

[63] *TensorFlow*. (n.d.). Google Open Source. https://opensource.google/projects/tensorflow

[64] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[65] OpenAI. (2023). *GPT-4 Technical Report*. https://openai.com/research/gpt-4

[66] Google. (2023). *Gemma: lightweight, state-of-the-art open models*. https://ai.google.dev/gemma

[67] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention is All You Need*. Advances in Neural Information Processing Systems. https://arxiv.org/abs/1706.03762